```
# ==== run.py ====
# -*- coding: utf-8 -*-
"""系统启动入口。
运行：
    python run.py                启动并自动打开浏览器
    python run.py --no-browser   只启动服务，不打开浏览器
    python run.py --dev          开发模式（开启代码热重载与调试器）
    python run.py --port 5001    指定端口
默认以 127.0.0.1:5000 启动，就绪后自动打开浏览器。
"""
import os
import sys
import threading
import time
import webbrowser
if hasattr(sys.stdout, "reconfigure"):
    sys.stdout.reconfigure(encoding="utf-8")   # 控制台中文正常显示
from app import create_app
from app.models import db
PORT = 5000
for i, a in enumerate(sys.argv):
    if a == "--port" and i + 1 < len(sys.argv):
        PORT = int(sys.argv[i + 1])
URL = "http://127.0.0.1:%d" % PORT
NO_BROWSER = "--no-browser" in sys.argv
DEV = "--dev" in sys.argv
app = create_app()
def open_browser_later(delay=1.2):
    """等服务工作起来再打开浏览器，避免打开时页面还没就绪。
    放在后台线程里，不用 sleep 阻塞服务启动；线程设为守护线程，
    用户 Ctrl+C 停止服务时不会残留。
    """
    def run():
        time.sleep(delay)
        try:
            webbrowser.open(URL)
        except Exception:
            pass            # 打开失败不影响服务本身
    t = threading.Thread(target=run, daemon=True)
    t.start()
if __name__ == "__main__":
    with app.app_context():
        db.create_all()     # 首次运行时自动建表
        from app.utils.schema import ensure_schema
        added = ensure_schema(db)     # 旧库补齐新增字段
        if added:
            print("已补充字段：%s" % "、".join(added))
    print("=" * 58)
    print("   历史遗留废弃矿山自然恢复适宜性评价系统  Ｖ１．０")
    print("=" * 58)
```

```
    print("   访问地址：%s" % URL)
    print("   演示账号：admin      密码：admin123")
    print("   停止运行：按 Ctrl+C，或直接关闭本窗口")
    print("=" * 58)
    if not NO_BROWSER and not DEV:
        print("   正在打开浏览器……")
    print()
    # 开发模式才启用热重载（重载会二次启动进程，故此时不自动开浏览器）
    if not NO_BROWSER and not DEV:
        open_browser_later()
    app.run(host="127.0.0.1", port=PORT, debug=DEV,
    use_reloader=DEV)
# ==== app/__init__.py ====
# -*- coding: utf-8 -*-
"""Flask 应用工厂。"""
from datetime import date
from flask import Flask
from flask_login import LoginManager
from flask_wtf.csrf import CSRFProtect, CSRFError
from app.config import Config
from app.models import db, User
from app.utils.constants import ROLE_LABELS
login_manager = LoginManager()
login_manager.login_view = "auth.login"
login_manager.login_message = "请先登录"
csrf = CSRFProtect()
@login_manager.user_loader
def load_user(user_id):
    return db.session.get(User, int(user_id))
def create_app(config_class=Config):
    app = Flask(__name__)
    app.config.from_object(config_class)
    db.init_app(app)
    login_manager.init_app(app)
    # CSRF 防护：WTF_CSRF_ENABLED 只对 FlaskForm 生效，页面上那些手写 POST
    表单
    # （账号增删改、项目 / 图斑删除等）以前是没校验的；同时模板里的 csrf_token()
    # 也需要它才会存在——少了这一行，保存过一次 AHP 权重后 /ahp/ 直接 500。
    csrf.init_app(app)
    # 模板里展示时间统一走 localdt：库里存 UTC，展示成本地时间
    from app.utils.timefmt import to_local
    app.jinja_env.filters["localdt"] = to_local
    # 注册视图（各模块路由）
    from app.views import auth, project, unit, standard,
    constraint, \
        ahp, indicator, evaluate, report, search
    app.register_blueprint(auth.bp)
    app.register_blueprint(project.bp)
    app.register_blueprint(unit.bp)
    app.register_blueprint(standard.bp)
```

```
app.register_blueprint(constraint.bp)
app.register_blueprint(ahp.bp)
app.register_blueprint(indicator.bp)
app.register_blueprint(evaluate.bp)
app.register_blueprint(report.bp)
app.register_blueprint(search.bp)
#  全局登录保护：除登录页与静态资源外，均需登录
from flask import request, redirect, url_for,
send_from_directory, flash
from flask_login import current_user, login_user,
logout_user
PUBLIC_ENDPOINTS = {"auth.login", "auth.register",
"static", "favicon"}
#  浏览器默认会请求根路径下的 /favicon.ico。
#  页面里虽已用 <link rel="icon"> 指向 static 目录，但出错页等场景仍会
#  回退到这个路径，补一条路由免得日志里一直刷 404。
@app.route("/favicon.ico")
def favicon():
    return send_from_directory(app.static_folder,
"favicon.ico",
                               mimetype="image/vnd.micro
soft.icon")
@app.before_request
def require_login():
    #  截图模式（仅用于生成说明书配图）：自动以管理员身份登录
    if app.config.get("SCREENSHOT_MODE"):
        if (request.endpoint not in ("auth.login",
"auth.register", "static")
                and not current_user.is_authenticated):
            admin = User.query.filter_by(role="admin").f
irst()
            if admin:
                login_user(admin)
        return None
    if request.endpoint in PUBLIC_ENDPOINTS or
request.endpoint is None:
        return None
    if not current_user.is_authenticated:
        return redirect(url_for("auth.login"))
    #  账号被停用后立即失效：不然已经登录的会话还能继续用下去
    if not current_user.active:
        logout_user()
        flash("该账号已停用，请联系管理员", "toast-error")
        return redirect(url_for("auth.login"))
    return None
#  侧边栏导航：按评价流程的顺序编排，分为「评价流程」与「系统管理」两组。
#  不使用模块编号，避免出现与实际流程不一致的代号。
#  每项带图标名（内联 SVG，见 templates/_macros.html 的 ICONS）
#  与可选的待办角标（该步骤还没做完的图斑数）。
FLOW_LINKS = [
```

```
        ("项目与矿山档案", "/project/", "folder"),
        ("政策与标准知识库", "/standard/", "book"),
        ("限制性指标判定", "/constraint/", "shield"),
        ("微地质单元", "/unit/", "layers"),
        ("指标体系与权重", "/ahp/", "sliders"),
        ("指标数据采集", "/indicator/", "database"),
        ("适宜性定量评价", "/evaluate/", "target"),
        ("报告与结果输出", "/report/", "file"),
    ]
    #  「用户与操作日志」已并入「个人设置」，不再是独立菜单，
    #  因此系统管理组只留一个入口。
    ADMIN_LINKS = [
        ("个人设置", "/profile", "user"),
    ]
    @app.context_processor
    def inject_nav():
        path = request.path or "/"
        def hit(prefix):
            """当前路径是否属于某个入口（/unit/ 也匹配 /unit/3）。"""
            return path.startswith(prefix.rstrip("/"))
        badges = {}
        if current_user.is_authenticated:
            from app.services import dashboard as dash
            badges = dash.nav_badges()
        def build(links):
            return [{"label": label, "href": href, "icon":
    icon,
                     "active": hit(href), "badge":
    badges.get(href, 0)}
                    for label, href, icon in links]
        groups = [
            {"title": "评价流程", "links": build(FLOW_LINKS)},
            {"title": "系统管理", "links": build(ADMIN_LINKS)},
        ]
        #  面包屑用的当前页名：按路径命中的入口取，未命中时给个通用名
        if path == "/":
            page_title = "工作台"
        elif path.startswith("/profile"):
            page_title = "个人设置"
        elif path.startswith(("/users", "/logs")):
            page_title = "账号与日志"
        elif path.startswith("/search"):
            page_title = "搜索结果"
        else:
            page_title = "工作台"
            for label, href, _icon in FLOW_LINKS:
                if hit(href):
                    page_title = label
                    break
        #  层级引导用：项目是归档单位，图斑才是评价对象。
```

```
    #  下游各页面按图斑组织数据，因此需要提示「哪些项目下还没有图斑」。
    from app.models import Project, MinePatch
    projects = Project.query.order_by(Project.id).all()
    return {
        "nav_groups": groups,
        "home_active": path == "/",
        "page_title": page_title,
        "today": date.today().strftime("%Y-%m-%d"),
        "search_q": request.args.get("q", ""),
        "role_labels": ROLE_LABELS,
        "project_count": len(projects),
        "patch_count": MinePatch.query.count(),
        "projects_without_patch": [p for p in projects
                                   if p.patches.count()
== 0],
    }
#  操作审计：统一在响应阶段记录成功的写操作（操作日志）
from app.services import audit
@app.after_request
def audit_writes(response):
    if request.method in ("POST", "PUT", "PATCH",
"DELETE"):
        audit.record(request.endpoint,
response.status_code)
    return response
#  错误页：给出可读提示，而不是默认的英文错误页
from flask import render_template
@app.errorhandler(403)
def forbidden(e):
    return render_template("error.html", code=403,
                           message="没有权限执行该操作"), 403
@app.errorhandler(404)
def not_found(e):
    return render_template("error.html", code=404,
                           message="请求的页面不存在"), 404
@app.errorhandler(413)
def too_large(e):
    """上传超过体积上限时给中文页；默认是英文的 Request Entity Too Large。"""
    limit = app.config.get("MAX_CONTENT_LENGTH") or 0
    return render_template(
        "error.html", code=413,
        message="上传的文件太大（上限 %d MB），请压缩或拆分后重试"
                % (limit // 1024 // 1024)), 413
@app.errorhandler(500)
def server_error(e):
    """未预料的错误也给中文页；生产模式下 Flask 不会带出堆栈。"""
    return render_template("error.html", code=500,
                           message="系统处理时出现异常，本次操作未生效，"
                                   "请返回重试或联系管理员"), 500
@app.errorhandler(CSRFError)
```

```
    def csrf_error(e):
        """令牌过期或来源不符时给一句人话，并把人送回上一页。
        原来校验一失败就是默认英文 400 页，或者（对手写表单）干脆什么都不发生，
        用户只会觉得「点了没反应」。回跳目标只取本站地址，避免开放重定向。
        """
        from flask import flash, redirect, url_for
        flash("页面停留时间过久或提交来源校验未通过，请重新提交一次", "error")
        referrer = request.referrer or ""
        if referrer.startswith(request.host_url):
            return redirect(referrer)
        return redirect(url_for("auth.index"))
    return app
# ==== app/config.py ====
# -*- coding: utf-8 -*-
"""应用配置。"""
import os
import secrets
from datetime import timedelta
BASE_DIR = os.path.abspath(os.path.dirname(__file__))
def _secret_key():
    """会话签名密钥：优先取环境变量，否则首次运行随机生成并存到本地文件。
    原来是一个写死在代码里的默认值，任何拿到源码的人都能自签会话 cookie。
    单机本地跑风险有限，但改成本机随机值几乎零成本。
    生成的文件已加进 .gitignore，不会进版本库。
    """
    env = os.environ.get("SECRET_KEY")
    if env:
        return env
    path = os.path.join(BASE_DIR, ".secret_key")
    try:
        if os.path.exists(path):
            with open(path, encoding="utf-8") as f:
                saved = f.read().strip()
            if saved:
                return saved
        key = secrets.token_hex(32)
        with open(path, "w", encoding="utf-8") as f:
            f.write(key)
        return key
    except OSError:
        # 目录不可写时退回固定值，保证系统仍能起来（此时会退化为原来的行为）
        return "mine-restoration-dev-key"
class Config:
    SECRET_KEY = _secret_key()
    # 单机部署，SQLite 数据库文件放在应用目录下
    SQLALCHEMY_DATABASE_URI = "sqlite:///" +
    os.path.join(BASE_DIR, "app.db")
    SQLALCHEMY_TRACK_MODIFICATIONS = False
    # 禁止跨站请求伪造（CSRF），登录与表单提交依赖此项
    WTF_CSRF_ENABLED = True
```

```
    # 会话 cookie：不交给脚本读取，跨站请求不带
    SESSION_COOKIE_HTTPONLY = True
    SESSION_COOKIE_SAMESITE = "Lax"
    # 模板改动立即生效，不必重启服务。
    # Flask 在非 debug 下默认把模板缓存到进程里，改完模板刷新浏览器还是旧页面，
    # 容易被误当成「改了没生效」。本系统只在本机运行，这点开销可以忽略。
    TEMPLATES_AUTO_RELOAD = True
    # 「记住我」：登录时勾选则下发持久 cookie，关闭浏览器后仍可自动登录
    REMEMBER_COOKIE_DURATION = timedelta(days=30)
    REMEMBER_COOKIE_HTTPONLY = True
    REMEMBER_COOKIE_SAMESITE = "Lax"
    # 影像与数据文件上传（遥感影像挂接、批量导入）
    UPLOAD_FOLDER = os.path.join(BASE_DIR, "static",
    "uploads")
    MAX_CONTENT_LENGTH = 32 * 1024 * 1024          # 单次上传上限
    32 MB
    ALLOWED_IMAGE_EXT = {"png", "jpg", "jpeg", "gif",
    "webp", "bmp", "tif", "tiff"}
    ALLOWED_DATA_EXT = {"csv", "xlsx", "xls"}
    # 头像只用浏览器能直接显示的格式（不含 bmp / tif），并单独限制体积
    ALLOWED_AVATAR_EXT = {"png", "jpg", "jpeg", "gif",
    "webp"}
    MAX_AVATAR_LENGTH = 4 * 1024 * 1024            # 4 MB
# ==== app/models.py ====
# -*- coding: utf-8 -*-
"""数据模型：系统全部数据表。
本文件由甲在阶段 0 一次性写完，作为两人独立开发的地基。
约定：两人只读此文件；确需改表时，由乙提需求、甲统一修改。
"""
import json
from flask_login import UserMixin
from flask_sqlalchemy import SQLAlchemy
from app.utils.timefmt import utcnow    # 替代已弃用的
    datetime.utcnow()
db = SQLAlchemy()
class User(UserMixin, db.Model):
    """用户（账户与权限）。"""
    __tablename__ = "users"
    id = db.Column(db.Integer, primary_key=True)
    username = db.Column(db.String(64), unique=True,
    nullable=False, index=True)
    password_hash = db.Column(db.String(256),
    nullable=False)
    # 职务只有两种：管理员 admin（可管理账号与他人的职务） / 操作员 operator
    role = db.Column(db.String(20), nullable=False,
    default="operator")
    active = db.Column(db.Boolean, nullable=False,
    default=True)
    created_at = db.Column(db.DateTime, default=utcnow)
    # ---- 个人资料（用户可在「个人设置」里自行维护）----
```

```
    display_name = db.Column(db.String(64), nullable=True)
    # 昵称
    email = db.Column(db.String(128), nullable=True)
    phone = db.Column(db.String(32), nullable=True)
    gender = db.Column(db.String(8), nullable=True)
    # 男 / 女 / 保密
    avatar = db.Column(db.String(256), nullable=True)
    # 头像，相对 static 的路径
    @property
    def is_admin(self):
        return self.role == "admin"
    @property
    def shown_name(self):
        """界面上显示的名字：优先用昵称，没填就用账号名。"""
        return self.display_name or self.username
    @property
    def initial(self):
        """没有头像时用首字占位。"""
        return (self.shown_name or "?")[0].upper()
    def to_dict(self):
        return {"id": self.id, "username": self.username,
                "display_name": self.display_name,
    "gender": self.gender,
                "role": self.role, "active": self.active}
class OperationLog(db.Model):
    """操作日志。"""
    __tablename__ = "operation_logs"
    id = db.Column(db.Integer, primary_key=True)
    user_id = db.Column(db.Integer, db.ForeignKey("users.id"
    ), nullable=True)
    action = db.Column(db.String(128), nullable=False)
    # 操作类型
    target_type = db.Column(db.String(64), nullable=True)
    # 对象类型
    target_id = db.Column(db.Integer, nullable=True)
    # 对象主键
    detail = db.Column(db.Text, nullable=True)
    # 操作详情
    created_at = db.Column(db.DateTime, default=utcnow,
    index=True)
class Project(db.Model):
    """评价项目（项目与矿山档案）。"""
    __tablename__ = "projects"
    id = db.Column(db.Integer, primary_key=True)
    name = db.Column(db.String(128), nullable=False)
    code = db.Column(db.String(64), nullable=True)
    # 项目编号
    description = db.Column(db.Text, nullable=True)
    status = db.Column(db.String(20), default="进行中")
    # 进行中 / 已完成 / 已归档
```

```
    created_by = db.Column(db.Integer,                                  401
    db.ForeignKey("users.id"), nullable=True)                           402
    created_at = db.Column(db.DateTime, default=utcnow)                 403
    patches = db.relationship("MinePatch",                              404
    backref="project", lazy="dynamic",                                  405
                                    cascade="all, delete-orphan")       406
class MinePatch(db.Model):                                              407
    """矿山图斑。"""                                                    408
    __tablename__ = "mine_patches"                                      409
    id = db.Column(db.Integer, primary_key=True)                        410
    project_id = db.Column(db.Integer,                                  411
    db.ForeignKey("projects.id"), nullable=False)                       412
    name = db.Column(db.String(128), nullable=False)                    413
    admin_region = db.Column(db.String(128), nullable=True)             414
    # 所在行政区                                                        415
    center_x = db.Column(db.Float, nullable=True)                       416
    # 中心经度 / 横坐标                                                 417
    center_y = db.Column(db.Float, nullable=True)                       418
    # 中心纬度 / 纵坐标                                                 419
    area_ha = db.Column(db.Float, nullable=True)                        420
    # 图斑面积（公顷）                                                  421
    mineral_type = db.Column(db.String(64), nullable=True)              422
    # 开采矿种                                                          423
    mining_method = db.Column(db.String(64), nullable=True)             424
    # 开采方式                                                          425
    stop_years = db.Column(db.Float, nullable=True)                     426
    # 停产年限                                                          427
    responsible_entity_missing = db.Column(db.Boolean,                  428
    default=True)   # 责任主体是否缺失                                 429
    # 遥感底图 / 无人机正射影像挂接                                     430
    image_path = db.Column(db.String(256), nullable=True)               431
    # 相对 static 的路径                                                432
    image_note = db.Column(db.String(256), nullable=True)               433
    # 影像说明（如来源、时相）                                          434
    image_uploaded_at = db.Column(db.DateTime,                          435
    nullable=True)                                                      436
    created_at = db.Column(db.DateTime, default=utcnow)                 437
    units = db.relationship("MicroUnit", backref="patch",               438
    lazy="dynamic",                                                     439
                                    cascade="all, delete-orphan")       440
class ReferenceEcosystem(db.Model):                                     441
    """参照生态系统：标准化基准值，与图斑绑定。"""                     442
    __tablename__ = "reference_ecosystems"                              443
    id = db.Column(db.Integer, primary_key=True)                        444
    patch_id = db.Column(db.Integer, db.ForeignKey("mine_pat            445
    ches.id"), nullable=False)                                          446
    coverage_benchmark = db.Column(db.Float, default=75.0)              447
    # 植被覆盖度基准 %                                                  448
    shannon_benchmark = db.Column(db.Float, default=0.7)                449
    # Shannon-Wiener 基准                                               450
```

```
    dominant_species = db.Column(db.String(256),                    451
    nullable=True)   # 建群种                                        452
    soil_benchmark = db.Column(db.String(256),                      453
    nullable=True)     # 土壤基准                                   454
class Standard(db.Model):                                           455
    """地方标准 / 政策文件（政策与标准知识库）。"""                    456
    __tablename__ = "standards"                                     457
    id = db.Column(db.Integer, primary_key=True)                    458
    province = db.Column(db.String(32), nullable=False)             459
    # 省份                                                          460
    name = db.Column(db.String(256), nullable=False)                461
    # 名称                                                          462
    doc_number = db.Column(db.String(128), nullable=True)           463
    # 标准编号 / 文号                                               464
    publish_date = db.Column(db.String(32), nullable=True)          465
    implement_date = db.Column(db.String(32), nullable=True)        466
    # 类别：地方标准 / 政策文件                                     467
    category = db.Column(db.String(32), nullable=False,             468
    default="地方标准")                                             469
    summary = db.Column(db.Text, nullable=True)                     470
    # 特点概述                                                      471
    rules = db.relationship("StandardRule",                         472
    backref="standard",                                             473
                                cascade="all, delete-orphan")       474
class StandardRule(db.Model):                                       475
    """结构化规则：将标准条文拆解为可执行规则。"""                    476
    __tablename__ = "standard_rules"                                477
    id = db.Column(db.Integer, primary_key=True)                    478
    standard_id = db.Column(db.Integer,                             479
    db.ForeignKey("standards.id"), nullable=False)                  480
    item = db.Column(db.String(128), nullable=False)                481
    # 判定项                                                        482
    operator = db.Column(db.String(8), nullable=False)              483
    # 比较符：> >= < <= ==                                          484
    threshold = db.Column(db.String(64), nullable=True)             485
    # 阈值                                                          486
    condition_type = db.Column(db.String(32),                       487
    default="必要条件")   # 必要条件 / 充分条件                     488
    region = db.Column(db.String(64), nullable=True)                489
    # 适用分区（宁夏分区差异化）                                    490
    note = db.Column(db.Text, nullable=True)                        491
class ConstraintCheck(db.Model):                                    492
    """限制性判定记录。"""                                          493
    __tablename__ = "constraint_checks"                             494
    id = db.Column(db.Integer, primary_key=True)                    495
    patch_id = db.Column(db.Integer, db.ForeignKey("mine_pat        496
    ches.id"), nullable=False)                                      497
    policy_pass = db.Column(db.Boolean, nullable=True)              498
    # 政策约束                                                      499
    location_pass = db.Column(db.Boolean, nullable=True)            500
```

```
    # 区位约束
    pollution_pass = db.Column(db.Boolean, nullable=True)
    # 土壤污染
    geo_pass = db.Column(db.Boolean, nullable=True)
    # 地质灾害
    farmland_pass = db.Column(db.Boolean, nullable=True)
    # 耕地属性
    passed = db.Column(db.Boolean, nullable=True)
    # 总体是否通过
    reason = db.Column(db.Text, nullable=True)
    # 不通过原因
    checked_at = db.Column(db.DateTime, default=utcnow)
class MicroUnit(db.Model):
    """微地质单元：岩质边坡 / 碎石边坡 / 平台。"""
    __tablename__ = "micro_units"
    id = db.Column(db.Integer, primary_key=True)
    patch_id = db.Column(db.Integer, db.ForeignKey("mine_pat
    ches.id"), nullable=False)
    unit_type = db.Column(db.String(32), nullable=False)
    # 单元类型
    area = db.Column(db.Float, nullable=True)
    # 面积
    ratio = db.Column(db.Float, nullable=True)
    # 面积占比（自动计算）
    description = db.Column(db.Text, nullable=True)
    # 现场描述
class Indicator(db.Model):
    """指标定义：指标树，含角色与启用单元类型。"""
    __tablename__ = "indicators"
    id = db.Column(db.Integer, primary_key=True)
    code = db.Column(db.String(32), unique=True,
    nullable=False)  # 指标编码
    name = db.Column(db.String(64), nullable=False)
    # 指标名称
    level = db.Column(db.Integer, nullable=False)
    # 1/2/3 级
    parent_code = db.Column(db.String(32), nullable=True)
    # 父指标编码
    # 角色：土壤污染指标同时作为限制性指标与评价指标
    is_constraint = db.Column(db.Boolean, default=False)
    is_evaluative = db.Column(db.Boolean, default=True)
    unit = db.Column(db.String(32), nullable=True)
    # 单位
    ideal_value = db.Column(db.String(64), nullable=True)
    # 理想值（可配置）
    ideal_type = db.Column(db.String(32), nullable=True)
    # benefit/cost/mid/discrete
    # 启用该指标的微地质单元（逗号分隔）。岩质边坡不启用土壤指标、平台不启用地形指标
    enabled_units = db.Column(db.String(128), nullable=True)
    description = db.Column(db.Text, nullable=True)
```

```
    # 取值依据说明
    def units_list(self):
        return [s for s in (self.enabled_units or
    "").split(",") if s]
class WeightScheme(db.Model):
    """权重方案：按微地质单元类型分别保存。"""
    __tablename__ = "weight_schemes"
    id = db.Column(db.Integer, primary_key=True)
    unit_type = db.Column(db.String(32), nullable=False)
    # 所属单元类型
    name = db.Column(db.String(64), nullable=True)
    # 方案名称
    level = db.Column(db.Integer, nullable=False,
    default=1)          # 1=一级指标 2=二级指标
    is_default = db.Column(db.Boolean, default=False)
    # 是否默认（系统默认）
    created_at = db.Column(db.DateTime, default=utcnow)
class WeightItem(db.Model):
    """权重明细：方案内的各项权重。"""
    __tablename__ = "weight_items"
    id = db.Column(db.Integer, primary_key=True)
    scheme_id = db.Column(db.Integer,
    db.ForeignKey("weight_schemes.id"), nullable=False)
    indicator_code = db.Column(db.String(32),
    nullable=False)        # 指标编码
    weight = db.Column(db.Float, nullable=False)
    # 权重值
class ExpertMatrix(db.Model):
    """专家判断矩阵：多专家打分的原始矩阵。"""
    __tablename__ = "expert_matrices"
    id = db.Column(db.Integer, primary_key=True)
    scheme_id = db.Column(db.Integer,
    db.ForeignKey("weight_schemes.id"), nullable=True)
    # 直接标注适用对象，便于无需先建权重方案即可录入专家矩阵
    unit_type = db.Column(db.String(32), nullable=True)
    # 微地质单元类型
    l1_name = db.Column(db.String(64), nullable=True)
    # 所属一级指标（二级矩阵用）
    expert_name = db.Column(db.String(64), nullable=True)
    # 专家姓名
    specialty = db.Column(db.String(64), nullable=True)
    # 专家专业
    level = db.Column(db.Integer, nullable=False,
    default=1)          # 矩阵对应指标层级
    matrix_json = db.Column(db.Text, nullable=False)
    # 判断矩阵（JSON 二维数组）
    cr = db.Column(db.Float, nullable=True)
    # 该专家矩阵的一致性比例
    created_at = db.Column(db.DateTime, default=utcnow)
class IndicatorData(db.Model):
```

```
    """指标实测数据：按微地质单元组织。"""
    __tablename__ = "indicator_data"
    id = db.Column(db.Integer, primary_key=True)
    unit_id = db.Column(db.Integer, db.ForeignKey("micro_uni
    ts.id"), nullable=False, unique=True)
    soil_ratio = db.Column(db.Float, nullable=True)
    # 土石比
    thickness = db.Column(db.Float, nullable=True)
    # 土层厚度 cm
    ph = db.Column(db.Float, nullable=True)
    organic_matter = db.Column(db.Float, nullable=True)
    # 有机质 g/kg
    # 重金属含量，JSON 字符串：{"Hg": 0.05, ...}，未检测项缺失或为 null
    heavy_metals = db.Column(db.Text, nullable=True)
    coverage = db.Column(db.Float, nullable=True)
    # 植被覆盖度 %
    shannon = db.Column(db.Float, nullable=True)
    # Shannon-Wiener 多样性指数
    slope = db.Column(db.Float, nullable=True)
    # 坡度 °
    aspect = db.Column(db.String(16), nullable=True)
    # 坡向：阳坡/阴阳坡/阴坡
    def heavy_metals_dict(self):
        """把重金属 JSON 解析成字典。
        字段是手工可改的文本，被脏写或导入半个 JSON 时不能让整页 500：
        解析失败按「未检测」处理（空字典），并把原文留在库里供排查。
        """
        if not self.heavy_metals:
            return {}
        try:
            data = json.loads(self.heavy_metals)
        except (ValueError, TypeError):
            return {}
        return data if isinstance(data, dict) else {}
class EvaluationResult(db.Model):
    """评价结果。"""
    __tablename__ = "evaluation_results"
    id = db.Column(db.Integer, primary_key=True)
    patch_id = db.Column(db.Integer, db.ForeignKey("mine_pat
    ches.id"), nullable=False)
    final_score = db.Column(db.Float, nullable=True)
    # 最终评价因子 F
    grade = db.Column(db.String(32), nullable=True)
    # 适宜性等级
    measures = db.Column(db.Text, nullable=True)
    # 推荐措施
    unit_scores = db.Column(db.Text, nullable=True)
    # 各单元得分 JSON
    # 计算参数快照：本次用的权重来源、参照生态系统基准、各指标理想取值。
    # 参数现在可以在界面上改，不留快照就无法复现这一次的结果。
```

```
    params_json = db.Column(db.Text, nullable=True)
    created_by = db.Column(db.Integer,
    db.ForeignKey("users.id"), nullable=True)
    created_at = db.Column(db.DateTime, default=utcnow)
class ScoreDetail(db.Model):
    """得分明细：每一层指标的权重、标准化值、加权贡献，供追溯与复核。"""
    __tablename__ = "score_details"
    id = db.Column(db.Integer, primary_key=True)
    result_id = db.Column(db.Integer,
    db.ForeignKey("evaluation_results.id"), nullable=False)
    unit_id = db.Column(db.Integer, nullable=True)
    l1_name = db.Column(db.String(64), nullable=True)
    # 一级指标
    l2_name = db.Column(db.String(64), nullable=True)
    # 二级指标
    raw_value = db.Column(db.String(64), nullable=True)
    # 实测值
    std_value = db.Column(db.Float, nullable=True)
    # 标准化值
    weight = db.Column(db.Float, nullable=True)
    # 权重
    contribution = db.Column(db.Float, nullable=True)
    # 加权贡献
# ==== app/forms.py ====
# -*- coding: utf-8 -*-
"""表单定义（共享，由甲统一维护）。
各模块的 Flask-WTF 表单集中于此。
"""
import re
from flask_wtf import FlaskForm
from wtforms import SelectField, SubmitField, StringField,
    PasswordField, BooleanField
from wtforms.validators import (DataRequired, Length,
    Optional, EqualTo,
                                ValidationError)
from app.utils.constants import UNIT_TYPES, ASPECT_TYPES,
    GENDERS
# 邮箱格式：够用的轻量校验。
# WTForms 自带的 Email() 需要额外安装 email_validator,
# 为一个字段引入依赖不划算，这里自己用正则判断。
EMAIL_RE = re.compile(r"^[^@\s]+@[^@\s]+\.[A-Za-z]{2,}$")
def email_format(form, field):
    v = (field.data or "").strip()
    if v and not EMAIL_RE.match(v):
        raise ValidationError("邮箱格式不正确")
def email_ok(value):
    """邮箱格式判断，供不用 WTForms 的表单复用（如管理员编辑用户）。"""
    v = (value or "").strip()
    return (not v) or bool(EMAIL_RE.match(v))
class AHPSelectForm(FlaskForm):
```

```
    """选择微地质单元类型与指标层级，以确定判断矩阵规模。"""
    unit_type = SelectField("微地质单元类型",
    validators=[DataRequired()])
    level = SelectField("指标层级",
                        choices=[("1", "一级指标"), ("2",
    "二级指标")],
                        validators=[DataRequired()])
    l1 = SelectField("一级指标", validators=[],
    validate_choice=False)  # 动态填充，层级=2 时使用
    submit = SubmitField("生成判断矩阵")
    def __init__(self, *args, **kwargs):
        super().__init__(*args, **kwargs)
        self.unit_type.choices = [(t, t) for t in
    UNIT_TYPES]
class EmptyForm(FlaskForm):
    """仅用于承载 CSRF 令牌的空表单（矩阵录入等自定义表单使用）。"""
    pass
class LoginForm(FlaskForm):
    """登录表单。支持「记住我」以在关闭浏览器后保持登录状态。
    校验消息用中文写明：登录框带 novalidate，浏览器不会代填提示，
    字段为空时若不自己给消息，用户点了「登录」会以为没反应。
    """
    username = StringField("账户名称",
                           validators=[DataRequired(message=
    "请输入账户名称"),
                                       Length(max=64,

    message="账户名称不能超过 64 个字符")])
    password = PasswordField("密码",
                             validators=[DataRequired(messag
    e="请输入密码")])
    remember = BooleanField("记住我")
    submit = SubmitField("登录")
GENDER_CHOICES = [("", "不填")] + [(g, g) for g in GENDERS]
class ProfileForm(FlaskForm):
    """个人资料：昵称、性别与联系方式。字段都可留空。
    职务不在这里 —— 只能由管理员在「个人设置 · 账号」里改。
    """
    display_name = StringField("昵称", validators=[Optional(),
    Length(max=64)])
    gender = SelectField("性别", choices=GENDER_CHOICES,
    validators=[Optional()])
    email = StringField("邮箱", validators=[Optional(),
    email_format(),
                                            Length(max=128)])
    phone = StringField("手机号", validators=[Optional(),
    Length(max=32)])
    submit_info = SubmitField("保存资料")
class RegisterForm(FlaskForm):
    """注册：账号与密码必填，其余选填。注册后默认是操作员。"""
```

```
    username = StringField("账号名称",                                    751
                           validators=[DataRequired("请填写账号名称      752
    "),                                                                   753
                                       Length(min=2, max=64,              754
                                              message="账号名称           755
    2-64 个字符")])                                                       756
    password = PasswordField("密码",                                      757
                             validators=[DataRequired("请设置密码         758
    "),                                                                   759
                                         Length(min=6,                    760
    max=64,                                                               761
                                                                          762
    message="密码至少 6 位")])                                            763
    confirm = PasswordField("确认密码",                                   764
                            validators=[DataRequired(),                   765
                                        EqualTo("password",               766
                                                                          767
    message="两次输入的密码不一致")])                                     768
    display_name = StringField("昵称", validators=[Optional(),            769
    Length(max=64)])                                                      770
    gender = SelectField("性别", choices=GENDER_CHOICES,                  771
    validators=[Optional()])                                              772
    email = StringField("邮箱", validators=[Optional(),                   773
    email_format,                                                         774
                                        Length(max=128)])                 775
    phone = StringField("手机号", validators=[Optional(),                 776
    Length(max=32)])                                                      777
    submit_reg = SubmitField("注册并登录")                                778
class PasswordForm(FlaskForm):                                            779
    """修改密码：需要先验证当前密码。"""                                  780
    old_password = PasswordField("当前密码",                              781
    validators=[DataRequired()])                                          782
    new_password = PasswordField("新密码",                                783
                                 validators=[DataRequired(),              784
    Length(min=6, max=64,                                                 785
                                                                          786
    message="新密码至少 6 位")])                                          787
    confirm = PasswordField("确认新密码",                                 788
                            validators=[DataRequired(),                   789
                                                                          790
    EqualTo("new_password",                                               791
                                                                          792
    message="两次输入的新密码不一致")])                                   793
    submit_pwd = SubmitField("修改密码")                                  794
# ==== app/utils/constants.py ====                                        795
# -*- coding: utf-8 -*-                                                   796
"""评价指标常量表。                                                       797
全部数值为系统内置值，作为计算的唯一基准：                                798
- 取值规定：指标标准化理想极值与取值依据                                  799
- 等级划分规定：适宜性等级划分                                            800
```

```
- 系统默认权重：按微地质单元分别确定的一级、二级指标权重
- 权重计算方法：平均随机一致性指标 ＲＩ
"""
# ----------------------------------------------------------------
    ----------
# 一、微地质单元类型
# ----------------------------------------------------------------
    ----------
# 流程约定："按照地形地貌划分，将其划分为平台、碎石边坡和岩质边坡
# 三种地质微单元类型"
ＵＮＩＴ＿ＴＹＰＥＳ ＝ （"岩质边坡"， "碎石边坡"， "平台"）
# 每个单元类型启用的指标类别（系统约定：岩质边坡无土壤指标，平台无地形指标）
# 供数据录入界面与评价引擎判断"该单元是否需要某类指标"
ＵＮＩＴ＿ＥＮＡＢＬＥＤ＿ＩＮＤＩＣＡＴＯＲＳ ＝ ｛
    "岩质边坡"： （"植被指标"， "地形指标"）， # 不设土壤指标
    "碎石边坡"： （"土壤指标"， "植被指标"， "地形指标"），
    "平台"： （"土壤指标"， "植被指标"），  # 不设地形指标
｝
# ----------------------------------------------------------------
    ----------
# 二、指标标准化理想极值（指标取值规定）
# ----------------------------------------------------------------
    ----------
ＩＤＥＡＬ＿ＳＯＩＬ＿ＲＡＴＩＯ ＝ ０．８ # 土石比理想值
ＩＤＥＡＬ＿ＴＨＩＣＫＮＥＳＳ ＝ ２０．０ # 土层厚度理想值 （ｃｍ）
ＩＤＥＡＬ＿ＰＨ＿ＲＡＮＧＥ ＝ （５．０， ９．０） # ｐＨ 理想区间（适中型指标）
ＩＤＥＡＬ＿ＯＲＧＡＮＩＣ ＝ ３０．０ # 有机质理想值
    （ｇ／ｋｇ），全国第二次土壤普查养分等级划分标准二等最低值
ＩＤＥＡＬ＿ＣＯＶＥＲＡＧＥ ＝ ７５．０ # 植被覆盖度参照生态系统 （％）
ＩＤＥＡＬ＿ＳＨＡＮＮＯＮ ＝ ０．７ # Ｓｈａｎｎｏｎ-Ｗｉｅｎｅｒ 多样性指数参照生态系统
# 重金属极值 （ｍｇ／ｋｇ），《绿化种植土壤》ＣＪ／Ｔ ３４０—２０１６ 土壤重金属含量 ＩＶ 级最高值
# 取值依据：镉、汞、铅、铬、砷、镍、铜、锌分别为 2、2、５３０、４００、４５、２２０、６００、８００
ＨＥＡＶＹ＿ＭＥＴＡＬＳ ＝ （"Ｃｄ"， "Ｈｇ"， "Ｐｂ"， "Ｃｒ"， "Ａｓ"， "Ｎｉ"， "Ｃｕ"，
    "Ｚｎ"）
ＨＥＡＶＹ＿ＭＥＴＡＬ＿ＬＩＭＩＴ ＝ ｛
    "Ｃｄ"： ２．０， "Ｈｇ"： ２．０， "Ｐｂ"： ５３０．０， "Ｃｒ"： ４００．０，
    "Ａｓ"： ４５．０， "Ｎｉ"： ２２０．０， "Ｃｕ"： ６００．０， "Ｚｎ"： ８００．０，
｝
# 污染指标下 8 种重金属"重要性相当，权重平均分配"（权重计算方法）
ＨＥＡＶＹ＿ＭＥＴＡＬ＿ＥＱＵＡＬ＿ＷＥＩＧＨＴ ＝ １．０ ／ ８．０
# 坡度分界，取值规定引《新一轮退耕还林还草总体方案》发改西部〔２０１４〕１７７２ 号
ＳＬＯＰＥ＿ＢＲＥＡＫ ＝ ２５．０
# 坡向离散赋值，取值依据："阳坡、阴阳坡、阴坡分别赋值 0．2、０．６、１"
ＡＳＰＥＣＴ＿ＳＣＯＲＥＳ ＝ ｛"阳坡"： ０．２， "阴阳坡"： ０．６， "阴坡"： １．０｝
ＡＳＰＥＣＴ＿ＴＹＰＥＳ ＝ （"阳坡"， "阴阳坡"， "阴坡"）
# ｐＨ 适中型标准化的容差边界（区间外线性衰减，避免得分突变）
ＰＨ＿ＴＯＬ＿ＬＯＷ ＝ ４．０
ＰＨ＿ＴＯＬ＿ＨＩＧＨ ＝ １０．０
# ----------------------------------------------------------------
    ----------
```

```
# 三、适宜性等级划分
# ------------------------------------------------------------
    ----------------
GRADE_LEVELS = (
    {"grade": "不适宜", "min": None, "max": 0.3,
     "meaning": "存在限制性条件或区域生态本底较差，自然恢复影响因子严重制约自然恢复",
     "measure": "生态重建、人工辅助"},
    {"grade": "基本适宜", "min": 0.3, "max": 0.7,
     "meaning": "生态本底良好，核心自然恢复影响因子基本满足自然恢复需求",
     "measure": "自然恢复为主，可辅以封闭场地和少量补种、补植、飞播等"},
    {"grade": "适宜", "min": 0.7, "max": None,
     "meaning": "各自然恢复影响因子均能满足自然恢复需求，自然演替速度快",
     "measure": "自然恢复"},
)
def grade_of(score):
    """按适宜性等级划分表判定等级。"""
    for g in GRADE_LEVELS:
        lo, hi = g["min"], g["max"]
        if lo is None and score < hi:
            return g
        if hi is None and score >= lo:
            return g
        if lo is not None and hi is not None and lo <=
    score < hi:
            return g
    # score 恰好等于 0.7 时落入"适宜"
    return GRADE_LEVELS[-1]
# ------------------------------------------------------------
    ----------------
# 四、平均随机一致性指标 RI（权重计算方法给出 n=2，3，4，5，其余为常用扩展值）
# ------------------------------------------------------------
    ----------------
RI_TABLE = {1: 0.0, 2: 0.0, 3: 0.58, 4: 0.90, 5: 1.12, 6:
    1.24,
            7: 1.32, 8: 1.41, 9: 1.45, 10: 1.49}
# 一致性比例阈值（方法要求：当 CR<0.1 时，判断矩阵满足一致性要求）
CR_THRESHOLD = 0.1
# ------------------------------------------------------------
    ----------------
# 五、默认权重方案（系统内置）
# ------------------------------------------------------------
    ----------------
# 指标取值规定："划分岩质边坡、碎石土边坡、平台三类微地质单元开展独立评价，
# 分别计算评价指标权重"（方法概述与流程均作"碎石边坡"，
# 系统统一采用"碎石边坡"）。权重由 10 位专家打分后经 AHP 方法确定。
# 下表为系统内置的归一化权重，作为系统默认方案。
DEFAULT_WEIGHTS = {
    "岩质边坡": {
        "l1": {"植被指标": 0.45, "地形指标": 0.55},
        "l2": {
```

```
                    "植被指标"： {"植被覆盖度"： 0.67， "生物多样性"： 0.33}，
                    "地形指标"： {"坡度"： 0.64， "坡向"： 0.36}，
                },
            },
            "碎石边坡"： {
                "l1"： {"土壤指标"： 0.44， "植被指标"： 0.31， "地形指标"： 0.25}，
                "l2"： {
                    "土壤指标"： {"土石比"： 0.25， "土层厚度"： 0.25， "pH"： 0.17，
                                "有机质"： 0.17， "污染指标"： 0.17}，
                    "植被指标"： {"植被覆盖度"： 0.64， "生物多样性"： 0.36}，
                    "地形指标"： {"坡度"： 0.67， "坡向"： 0.33}，
                },
            },
            "平台"： {
                "l1"： {"土壤指标"： 0.54， "植被指标"： 0.46}，
                "l2"： {
                    "土壤指标"： {"土石比"： 0.25， "土层厚度"： 0.25， "pH"： 0.17，
                                "有机质"： 0.17， "污染指标"： 0.17}，
                    "植被指标"： {"植被覆盖度"： 0.61， "生物多样性"： 0.39}，
                },
            },
        }
# ------------------------------------------------------------------------
    ------------------
# 六、限制性指标（指标清单与评价流程）
# ------------------------------------------------------------------------
    ------------------
# 方法概述与评价流程：四项限制性指标；流程段另明确"是否为耕地"为约束条件
CONSTRAINT_ITEMS = ("政策"， "区位约束"， "土壤污染"， "地质灾害"， "耕地属性")
# ------------------------------------------------------------------------
    ------------------
# 七、账号职务
# ------------------------------------------------------------------------
    ------------------
# 只分两种：管理员可管理账号、维护基础数据并查看操作日志，操作员用业务功能。
ROLE_LABELS = {"admin": "管理员"， "operator": "操作员"}
ROLE_CODES = ("admin"， "operator")
DEFAULT_ROLE = "operator"                # 新注册账号的默认职务
ROLE_ORDER = ("admin"， "operator")   # 列表排序用：管理员在前
# 性别（注册与个人资料中选填）
GENDERS = ("男"， "女"， "保密")
# ==== app/utils/schema.py ====
# -*- coding: utf-8 -*-
"""轻量结构迁移。
SQLite 下 `db.create_all()` 只建缺失的表，不会给已有表补列。
本模块在启动时检查目标列是否存在，缺失则 ALTER TABLE 补上，
使旧数据库无需重建即可用上新字段。
"""
from sqlalchemy import text
# （表名， 列名， 列定义）
```

```
REQUIRED_COLUMNS = [
    # 遥感影像挂接
    ("mine_patches", "image_path", "VARCHAR(256)"),
    ("mine_patches", "image_note", "VARCHAR(256)"),
    ("mine_patches", "image_uploaded_at", "DATETIME"),
    # 指标维护
    ("indicators", "enabled_units", "VARCHAR(128)"),
    ("indicators", "description", "TEXT"),
    # 多专家矩阵
    ("expert_matrices", "unit_type", "VARCHAR(32)"),
    ("expert_matrices", "l1_name", "VARCHAR(64)"),
    ("expert_matrices", "specialty", "VARCHAR(64)"),
    ("expert_matrices", "cr", "FLOAT"),
    # 用户个人资料（账户页）
    ("users", "display_name", "VARCHAR(64)"),
    ("users", "email", "VARCHAR(128)"),
    ("users", "phone", "VARCHAR(32)"),
    ("users", "gender", "VARCHAR(8)"),
    ("users", "avatar", "VARCHAR(256)"),
    # 评价结果的可追溯信息：计算参数快照与计算人
    ("evaluation_results", "params_json", "TEXT"),
    ("evaluation_results", "created_by", "INTEGER"),
]
# 职务体系由「管理员 / 评价员 / 查看者」三种收敛为「管理员 / 操作员」两种，
# 旧数据里的后两种统一归为操作员。
ROLE_MIGRATION = {"evaluator": "operator", "viewer":
    "operator"}
def _add_columns(db):
    added = []
    with db.engine.connect() as conn:
        for table, column, ddl in REQUIRED_COLUMNS:
            rows = conn.execute(text("PRAGMA
    table_info(%s)" % table)).fetchall()
            if not rows:
                continue                          #
    表还不存在，create_all 会建
            existing = {r[1] for r in rows}
            if column in existing:
                continue
            conn.execute(text(
                "ALTER TABLE %s ADD COLUMN %s %s" % (table,
    column, ddl)))
            conn.commit()
            added.append("%s.%s" % (table, column))
    return added
def _migrate_roles(db):
    """把旧角色名统一为新的两种，返回改动的行数。"""
    changed = 0
    with db.engine.connect() as conn:
        rows = conn.execute(text("PRAGMA
```

```
    table_info(users)")).fetchall()
        if not rows:
            return 0
        for old, new in ROLE_MIGRATION.items():
            r = conn.execute(text("UPDATE users SET role =
    :new "
                                  "WHERE role = :old"),
                                {"new": new, "old": old})
            changed += r.rowcount or 0
        conn.commit()
    return changed
def ensure_schema(db):
    """为已有表补齐缺失列，并做必要的数据迁移。返回变更清单。"""
    try:
        changed = _add_columns(db)
        n = _migrate_roles(db)
        if n:
            changed.append("用户角色归一为管理员/操作员 %d 条" % n)
        return changed
    except Exception as e:                                  #
    迁移失败不应阻断启动
        print("[schema] 结构检查跳过：%s" % e)
        return []
# ==== app/utils/uploads.py ====
# -*- coding: utf-8 -*-
"""上传文件处理（遥感影像、批量导入）。"""
import os
import re
import uuid
from datetime import datetime
from werkzeug.utils import secure_filename
def _safe_ext(filename):
    return os.path.splitext(filename)[1].lower().lstrip(".")
def ext_allowed(filename, allowed):
    return _safe_ext(filename) in allowed
def save_upload(file_storage, subdir, allowed_ext,
    upload_root):
    """保存上传文件，返回相对 static 的路径（如 uploads/patches/xxx.png）。
    文件名重命名为「时间戳_随机串.扩展名」，避免中文名与重名问题。
    """
    if file_storage is None or not file_storage.filename:
        return None, "未选择文件"
    ext = _safe_ext(file_storage.filename)
    if ext not in allowed_ext:
        return None, "不支持的文件类型：.%s" % ext
    target_dir = os.path.join(upload_root, subdir)
    os.makedirs(target_dir, exist_ok=True)
    # 仅保留安全字符，并加时间戳与随机串防重名
    base = secure_filename(os.path.splitext(file_storage.fil
    ename)[0]) or "file"
```

```
    base = re.sub(r"[^A-Za-z0-9_\-]", "", base)[:40] or "file"
    name = "%s_%s.%s" % (datetime.now().strftime("%Y%m%d%H%M%S"),
                         uuid.uuid4().hex[:6], ext)
    abs_path = os.path.join(target_dir, name)
    file_storage.save(abs_path)
    rel = os.path.join("uploads", subdir, name).replace("\\", "/")
    return rel, None
def remove_upload(rel_path, upload_root):
    """删除已保存的上传文件（相对 static 的路径）。
    路径必须落在上传根目录之内：rel_path 来自数据库，正常都是 save_upload
    生成的，但万一被改脏（手工改库、导入异常），`../` 这种值会删掉上传根目录
    之外的文件。这里用 realpath 做一次围栏校验。
    """
    if not rel_path:
        return
    root = os.path.realpath(upload_root)
    abs_path = os.path.realpath(
        os.path.join(root, rel_path.replace("uploads/", "", 1)))
    if abs_path != root and not abs_path.startswith(root + os.sep):
        return
    try:
        if os.path.isfile(abs_path):
            os.remove(abs_path)
    except OSError:
        pass
# ==== app/utils/charts.py ====
# -*- coding: utf-8 -*-
"""轻量图表。
用纯 SVG 输出，不依赖任何绘图库或前端脚本：
- 服务端渲染，页面与打印（PDF）中都能稳定显示
- 每个图元都带数值标签，黑白打印时仍可读（不单靠颜色区分）
- 单色方案，符合界面「一个视图一个强调色」的约定
"""
from html import escape
ACCENT = "#559FCF"
ACCENT_SOFT = "#64BBE0"
INK = "#23262b"
MUTED = "#6b7280"
LINE = "#dfe3e8"
# 科研顶刊配色
GRAYS = ["#6FAE47", "#FCE5D6", "#FFF2CC", "#DAE3F2",
         "#E9C9C9", "#559FCF"]
def _fmt(v, digits=4):
    try:
        return ("%%.%df" % digits) % float(v)
```

```
    except (TypeError, ValueError):                                    1101
        return str(v)                                                  1102
def hbars(items, width=620, bar_h=22, gap=12, label_w=150,             1103
    value_w=76):                                                       1104
    """水平条形图。items: [(标签, 数值, 显示文本)]"""                    1105
    if not items:                                                      1106
        return '<p class="muted">暂无数据</p>'                          1107
    n = len(items)                                                     1108
    height = n * (bar_h + gap) + 10                                    1109
    max_v = max((abs(float(v)) for _, v, _ in items),                  1110
    default=1) or 1                                                    1111
    body_w = width - label_w - value_w                                 1112
    parts = ['<svg viewBox="0 0 %d %d" width="100%%"                   1113
    height="%d" '                                                      1114
             'role="img" aria-label="条形图" '                         1115
             'style="max-width:%dpx;font-family:inherit;">'            1116
    % (width, height, height, width)]                                  1117
    for i, (label, value, disp) in enumerate(items):                   1118
        y = i * (bar_h + gap) + 5                                      1119
        frac = (abs(float(value)) / max_v) if max_v else 0             1120
        w = max(2.0, body_w * frac)                                    1121
        fill = GRAYS[i % len(GRAYS)]                                   1122
        parts.append(                                                  1123
            '<text x="%d" y="%d" text-anchor="end"                     1124
    font-size="12" fill="%s">%s</text>'                                1125
            % (label_w - 10, y + bar_h * 0.72, INK,                    1126
    escape(str(label))))                                               1127
        parts.append(                                                  1128
            '<rect x="%d" y="%d" width="%.1f" height="%d"              1129
    rx="3" fill="%s"/>'                                                1130
            % (label_w, y, w, bar_h, fill))                            1131
        parts.append(                                                  1132
            '<text x="%d" y="%d" font-size="12" fill="%s" '            1133
            'style="font-variant-numeric:tabular-nums;">%s</           1134
    text>'                                                             1135
            % (label_w + w + 8, y + bar_h * 0.72, MUTED,               1136
    escape(str(disp))))                                                1137
    parts.append('</svg>')                                             1138
    return "".join(parts)                                              1139
def donut(items, size=210, thickness=34):                              1140
    """环形图（权重分布）。items: [(标签, 数值, 显示文本)]              1141
    图例中给出数值与占比，黑白打印时靠图例与灰度深浅区分。              1142
    """                                                                1143
    vals = [(l, float(v), d) for l, v, d in items if                   1144
    float(v) > 0]                                                      1145
    if not vals:                                                       1146
        return '<p class="muted">暂无数据</p>'                          1147
    total = sum(v for _, v, _ in vals) or 1                            1148
    r = (size - thickness) / 2                                         1149
    cx = cy = size / 2                                                 1150
```

```
import math
parts = ['<svg viewBox="0 0 %d %d" width="%d"
height="%d" role="img" '
         'aria-label="权重分布环形图">' % (size, size, size,
size)]
angle = -math.pi / 2
for i, (label, v, disp) in enumerate(vals):
    sweep = 2 * math.pi * (v / total)
    a0, a1 = angle, angle + sweep
    x0, y0 = cx + r * math.cos(a0), cy + r *
math.sin(a0)
    x1, y1 = cx + r * math.cos(a1), cy + r *
math.sin(a1)
    large = 1 if sweep > math.pi else 0
    parts.append(
        '<path d="M %.2f %.2f A %.2f %.2f 0 %d 1 %.2f
%.2f" '
        'fill="none" stroke="%s" stroke-width="%d"/>'
        % (x0, y0, r, r, large, x1, y1, GRAYS[i %
len(GRAYS)], thickness))
    angle = a1
parts.append('<text x="%d" y="%d" text-anchor="middle"
font-size="12" '
             'fill="%s">合计</text>' % (cx, cy - 2,
MUTED))
parts.append('<text x="%d" y="%d" text-anchor="middle"
font-size="15" '
             'fill="%s" style="font-variant-numeric:tabu
lar-nums;">%s</text>'
             % (cx, cy + 18, INK, _fmt(total, 4)))
parts.append('</svg>')
legend = ['<table style="width:auto;margin-top:10px;"><t
head><tr>'
          '<th scope="col">项目</th><th
scope="col">数值</th>'
          '<th scope="col">占比</th></tr></thead><tbody>']
for i, (label, v, disp) in enumerate(vals):
    legend.append(
        '<tr><td style="text-align:left;">'
        '<span style="display:inline-block;width:10px;he
ight:10px;'
        'border-radius:2px;background:%s;margin-right:6p
x;"></span>%s</td>'
        '<td class="mono">%s</td><td
class="mono">%.1f%%</td></tr>'
        % (GRAYS[i % len(GRAYS)], escape(str(label)),
escape(str(disp)),
           100.0 * v / total))
legend.append('</tbody></table>')
return ('<div style="display:flex;flex-wrap:wrap;gap:20p
```

```
    x;align-items:center;">'                                                1201
            + "".join(parts) + "".join(legend) + '</div>')                  1202
# ==== app/utils/browser.py ====                                            1203
# -*- coding: utf-8 -*-                                                     1204
"""HTML 转 PDF（报告导出）。                                                1205
复用本机已安装的 Chrome/Edge 无头模式打印，避免引入 reportlab、             1206
weasyprint 等额外依赖。生成过程使用系统临时目录中的 ASCII 文件名，          1207
避免中文路径在 file:// URL 下出问题。                                       1208
"""                                                                         1209
import os                                                                   1210
import shutil                                                               1211
import subprocess                                                           1212
import tempfile                                                             1213
import uuid                                                                 1214
CANDIDATES = [                                                              1215
    r"C:\Program Files\Google\Chrome\Application\chrome.exe"                 1216
    ,                                                                       1217
    r"C:\Program Files (x86)\Google\Chrome\Application\chrom                 1218
    e.exe",                                                                 1219
    r"C:\Program Files (x86)\Microsoft\Edge\Application\msed                 1220
    ge.exe",                                                                1221
    r"C:\Program Files\Microsoft\Edge\Application\msedge.exe                 1222
    ",                                                                      1223
]                                                                           1224
# 允许通过环境变量覆盖，便于在其他机器上指定浏览器                          1225
ENV_KEY = "DSH_BROWSER_PATH"                                                1226
def browser_path():                                                         1227
    """定位可用的浏览器可执行文件，找不到返回 None。"""                     1228
    override = os.environ.get(ENV_KEY)                                      1229
    if override and os.path.isfile(override):                               1230
        return override                                                     1231
    for p in CANDIDATES:                                                    1232
        if os.path.isfile(p):                                               1233
            return p                                                        1234
    for name in ("chrome", "msedge", "chromium"):                           1235
        found = shutil.which(name)                                          1236
        if found:                                                           1237
            return found                                                    1238
    return None                                                             1239
def html_to_pdf(html, timeout=90):                                          1240
    """把 HTML 字符串渲染为 PDF，返回字节；失败返回 (None, 原因)。"""       1241
    exe = browser_path()                                                    1242
    if exe is None:                                                         1243
        return None, ("未找到可用的浏览器（Chrome/Edge），无法生成 PDF。"    1244
                      "可设置环境变量 %s 指向浏览器可执行文件。" % ENV_KEY)  1245
    tmpdir = tempfile.mkdtemp(prefix="dshpdf_")                             1246
    tag = uuid.uuid4().hex[:8]                                              1247
    html_path = os.path.join(tmpdir, "r_%s.html" % tag)                     1248
    pdf_path = os.path.join(tmpdir, "r_%s.pdf" % tag)                       1249
    try:                                                                    1250
```

```
            with open(html_path, "w", encoding="utf-8") as f:
                f.write(html)
            cmd = [
                exe, "--headless=new", "--disable-gpu",
        "--no-sandbox",
                "--no-first-run", "--no-default-browser-check",
                "--no-pdf-header-footer",
                "--virtual-time-budget=4000",
                "--print-to-pdf=%s" % pdf_path,
                "file:///" + html_path.replace("\\", "/"),
            ]
            p = subprocess.run(cmd, capture_output=True,
        text=True,
                               timeout=timeout,
        encoding="utf-8", errors="replace")
            if not os.path.isfile(pdf_path) or
        os.path.getsize(pdf_path) < 800:
                return None, "浏览器未能生成 PDF（%s）" % (p.stderr or
        "").strip()[:200]
            with open(pdf_path, "rb") as f:
                return f.read(), None
        except subprocess.TimeoutExpired:
            return None, "生成 PDF 超时，请稍后重试"
        except Exception as e:
            return None, "生成 PDF 失败：%s" % e
        finally:
            shutil.rmtree(tmpdir, ignore_errors=True)
# ==== app/services/ahp_service.py ====
# -*- coding: utf-8 -*-
"""AHP 权重引擎。
按权重计算方法实现：
- 式（２）（３）（４）：特征根法求权重（行元素乘积 → n 次方根 → 归一化）
- 式（５）（６）（７）：一致性检验（λmax、CI、CR）
- 方法要求 CR < 0.1；RI 按阶数查表
"""
import numpy as np
from app.utils.constants import RI_TABLE, CR_THRESHOLD
def matrix_from_upper(upper):
    """由判断矩阵上三角构造完整互反矩阵。
    参数
    ----
    upper : list[list[float]] 或 np.ndarray，上三角元素（不含对角线）。
        例：3 阶时传入 [[a12, a13], [a23]]。
        对角线为 1，下三角取倒数（a_ji = 1/a_ij）。
    """
    upper = np.asarray(upper, dtype=float)
    k = upper.shape[0]
    n = int(round((1 + (1 + 8 * k) ** 0.5) / 2))
    A = np.ones((n, n), dtype=float)
    idx = 0
```

```
    for i in range(n):
        for j in range(i + 1, n):
            A[i, j] = upper[idx]
            A[j, i] = 1.0 / upper[idx]
            idx += 1
    return A
def weights_and_consistency(A):
    """由判断矩阵 A 计算权重与一致性指标。
    返回
    ----
    dict：{weights: np.ndarray, lambda_max, ci, cr, n,
    consistent: bool}
    """
    A = np.asarray(A, dtype=float)
    n = A.shape[0]
    # 式(2) 行元素乘积
    row_prod = np.prod(A, axis=1)
    # 式(3) 开 n 次方根
    root = row_prod ** (1.0 / n)
    # 式(4) 归一化
    weights = root / root.sum()
    # 式(5) 最大特征值 λmax = (1/n) Σ (A·W)_i / W_i
    aw = A.dot(weights)
    lambda_max = float(np.sum(aw / weights) / n)
    # 式(6) 一致性指标 CI
    ci = (lambda_max - n) / (n - 1) if n > 1 else 0.0
    # 式(7) 一致性比例 CR = CI / RI
    ri = RI_TABLE.get(n, 1.49)
    cr = ci / ri if ri else 0.0
    return {
        "weights": weights,
        "lambda_max": lambda_max,
        "ci": ci,
        "cr": cr,
        "n": n,
        "consistent": cr < CR_THRESHOLD,
    }
def find_most_inconsistent(A):
    """定位判断矩阵中最不一致的一对比较。
    依据：完全一致矩阵应满足 a_ij = w_i / w_j。
    计算每个上三角元素与其理想值的相对偏差，返回偏差最大者。
    返回
    ----
    dict：{i, j, value, expected, deviation}，偏差最大的 (i, j)
    及对应取值。
    """
    A = np.asarray(A, dtype=float)
    r = weights_and_consistency(A)
    w = r["weights"]
    n = A.shape[0]
```

```
    best = None                                                    1351
    for i in range(n):                                             1352
        for j in range(i + 1, n):                                  1353
            expected = w[i] / w[j]                                 1354
            if expected == 0:                                      1355
                continue                                           1356
            deviation = abs(A[i, j] - expected) / expected         1357
            if best is None or deviation >                         1358
    best["deviation"]:                                             1359
                best = {"i": i, "j": j, "value": A[i, j],          1360
                        "expected": expected, "deviation":         1361
    deviation}                                                     1362
    return best                                                    1363
def aggregate_matrices(matrices):                                  1364
    """多专家判断矩阵聚合：几何平均（保持互反性）。                1365
    参数                                                           1366
    ----                                                           1367
    matrices : list[np.ndarray]，多个同阶判断矩阵。                1368
    """                                                            1369
    stack = np.stack([np.asarray(m, dtype=float) for m in          1370
    matrices])                                                     1371
    return np.prod(stack, axis=0) ** (1.0 / len(stack))            1372
def judge_from_weights(weights):                                   1373
    """由权重向量构造完全一致判断矩阵并检验。                      1374
    用途：验证给定权重向量能否作为某个完全一致判断矩阵的主特征向量 1375
    （A_ij = w_i / w_j 时 λmax = n、CR = 0）。                      1376
    """                                                            1377
    w = np.asarray(weights, dtype=float)                           1378
    n = w.shape[0]                                                 1379
    A = np.outer(w, 1.0 / w)                                       1380
    r = weights_and_consistency(A)                                 1381
    r["matrix"] = A                                                1382
    return r                                                       1383
# ==== app/services/standardization.py ====                        1384
# -*- coding: utf-8 -*-                                            1385
"""指标标准化（的一部分）。                                        1386
按标准化公式实现：                                                 1387
    x' = (x - x_min) / (x_max - x_min)                             1388
按指标性质分为四类：越大越优 / 越小越优 / 适中 / 离散赋值。        1389
所有理想极值取自 app.utils.constants，可配置。                     1390
"""                                                                1391
from app.utils.constants import (                                  1392
    IDEAL_SOIL_RATIO, IDEAL_THICKNESS, IDEAL_PH_RANGE,             1393
    IDEAL_ORGANIC,                                                 1394
    IDEAL_COVERAGE, IDEAL_SHANNON, HEAVY_METAL_LIMIT,              1395
    HEAVY_METAL_EQUAL_WEIGHT, SLOPE_BREAK, ASPECT_SCORES,          1396
    PH_TOL_LOW, PH_TOL_HIGH,                                       1397
)                                                                  1398
def std_benefit(value, ideal):                                     1399
    """越大越优指标标准化：以理想值为上限，按比例给分，饱和于 [0，1]。"""  1400
```

```
    if value is None:                                                    1401
        return None                                                      1402
    return max(0.0, min(1.0, value / ideal))                             1403
def std_cost(value, extreme):                                            1404
    """越小越优指标标准化：以极值为参考，线性折减至 [0,1]。"""                1405
    if value is None:                                                    1406
        return None                                                      1407
    return max(0.0, min(1.0, 1.0 - value / extreme))                     1408
def std_mid_ph(value, lo, hi, tol_lo=PH_TOL_LOW,                         1409
    tol_hi=PH_TOL_HIGH):                                                 1410
    """适中型指标标准化（pH）：理想区间内取 1，区间外线性衰减。"""           1411
    if value is None:                                                    1412
        return None                                                      1413
    if lo <= value <= hi:                                                1414
        return 1.0                                                       1415
    if value > hi:                                                       1416
        return max(0.0, (tol_hi - value) / (tol_hi - hi))                1417
    return max(0.0, (value - tol_lo) / (lo - tol_lo))                    1418
def std_slope(value, break_point=SLOPE_BREAK, upper=90.0):               1419
    """坡度标准化。                                                      1420
    取值规定：以 25° 为陡坡重要分界。25° 及以下记满分 1，                1421
    超过 25° 线性折减，90° 记 0。                                        1422
    """                                                                  1423
    if value is None:                                                    1424
        return None                                                      1425
    if value <= break_point:                                             1426
        return 1.0                                                       1427
    return max(0.0, (upper - value) / (upper - break_point))             1428
def aspect_score(aspect):                                                1429
    """坡向离散赋值：阳坡 0.2 / 阴阳坡 0.6 / 阴坡 1.0（取值规定）。"""     1430
    if aspect is None:                                                   1431
        return None                                                      1432
    return ASPECT_SCORES.get(aspect)                                     1433
def pollution_score(heavy_metals, limits=None):                          1434
    """污染指标：8 种重金属等权（各 0.125）合成。                        1435
    方法约定：未检测 Cr、Zn 的，从矿山类型看未受污染，按无污染（=1）处理。  1436
    heavy_metals : dict，键为金属名，值为含量 mg/kg；缺失键视为未检测。    1437
    limits : 可选，金属 → 极值；不传则用代码常量（界面改过就传库里的值）。  1438
    """                                                                  1439
    table = limits or HEAVY_METAL_LIMIT                                  1440
    total = 0.0                                                          1441
    for metal, extreme in table.items():                                 1442
        value = heavy_metals.get(metal)                                  1443
        total += std_cost(value, extreme) if value is not                1444
    None else 1.0                                                        1445
    return total / len(table)                                            1446
def standardize_unit(unit_type, data, weights=None,                      1447
    settings=None):                                                      1448
    """对单个微地质单元的全部指标做标准化。                              1449
    参数                                                                 1450
```

```
    ----                                                                1451
    unit_type : str，岩质边坡 / 碎石边坡 / 平台                          1452
    data : dict，实测数据，字段见接口契约：                              1453
        soil_ratio, thickness, ph, organic_matter,                      1454
    heavy_metals(dict),                                                 1455
        coverage, shannon, slope, aspect                                1456
    weights : dict，该单元类型的权重（app.utils.constants.DEFAULT_WEIG  1457
    HTS 结构），                                                        1458
                未提供时用默认值。                                      1459
    settings : app.services.unit_config.UnitSettings，图斑级的基准值   1460
    与理想取值。                                                        1461
                不传则全部按代码常量 —— 历史数据与验证算例走这条路径，结果不变。  1462
    返回                                                                1463
    ----                                                                1464
    dict：{ 一级指标： { 二级指标： （实测值， 标准化值， 权重， 贡献） } }  1465
    标准化值可能为 None，表示该指标不适用或方法待定。                   1466
    """                                                                 1467
    from app.utils.constants import DEFAULT_WEIGHTS                     1468
    from app.services import unit_config as uc                          1469
    if weights is None:                                                 1470
        weights = DEFAULT_WEIGHTS[unit_type]                            1471
    if settings is None:                                                1472
        settings = uc.UnitSettings()                                    1473
    # 理想取值：库里有数字就用库里的，否则用常量                        1474
    def ideal(code, fallback):                                          1475
        return settings.ideal(code, fallback)                           1476
    l1_enabled = weights["l1"].keys()                                   1477
    l2w = weights["l2"]                                                 1478
    result = {}                                                         1479
    # 一级指标是否启用：库里配过就按库里的（界面上的「启用单元」），没配按常量  1480
    def group_on(l1_name, l1_code):                                     1481
        return settings.group_enabled(unit_type, l1_name,               1482
    l1_code) is not False                                               1483
    # ---- 土壤指标 ----                                               1484
    if "土壤指标" in l1_enabled and group_on("土壤指标", "soil"):       1485
        result["土壤指标"] = {                                          1486
            "土石比": (data.get("soil_ratio"),                          1487
                       std_benefit(data.get("soil_ratio"),              1488
                                   ideal("soil_ratio",                  1489
    IDEAL_SOIL_RATIO)),                                                 1490
                       l2w["土壤指标"]["土石比"]),                      1491
            "土层厚度": (data.get("thickness"),                         1492
                         std_benefit(data.get("thickness"),             1493
                                     ideal("thickness",                 1494
    IDEAL_THICKNESS)),                                                  1495
                         l2w["土壤指标"]["土层厚度"]),                  1496
            "pH": (data.get("ph"),                                      1497
                   std_mid_ph(data.get("ph"),                           1498
    *IDEAL_PH_RANGE),                                                   1499
                   l2w["土壤指标"]["pH"]),                              1500
```

```
ok("project" in (last.target_type or ""), "日志记录了对象类型",
    last.target_type)
r = c.get("/profile?tab=logs")
body = r.get_data(as_text=True)
ok(r.status_code == 200 and "操作日志" in body, "操作日志分类可访问")
ok("新建项目" in body, "日志页显示操作记录")
# 账号编辑 / 删除（职务只有管理员与操作员两种）
with app.app_context():
    u = User(username="tmp_user", role="operator",
             password_hash=User.query.first().password_hash)
    db.session.add(u); db.session.commit()
    uid = u.id
r = c.get("/users/%d/edit" % uid)
ok(r.status_code == 200, "账号编辑页可访问")
# 保护校验要趁系统里只有一名管理员时做：
# 先降级唯一的 admin 应被拒绝，再把临时账号提为管理员看修改是否生效
with app.app_context():
    admin_id = User.query.filter_by(role="admin").first().id
c.post("/users/%d/edit" % admin_id,
       data={"role": "operator", "active": "1"})
with app.app_context():
    ok(db.session.get(User, admin_id).role == "admin",
    "拒绝把最后一个管理员降级")
c.post("/users/%d/edit" % uid, data={"role": "admin",
    "active": "1"})
with app.app_context():
    ok(db.session.get(User, uid).role == "admin",
    "职务已修改并保存")
c.post("/users/%d/delete" % uid, follow_redirects=True)
with app.app_context():
    ok(db.session.get(User, uid) is None, "账号删除生效")
# 清理日志核对用的项目
with app.app_context():
    for p in Project.query.filter_by(name="日志核对项目").all():
        db.session.delete(p)
    db.session.commit()
# ------------------------------------------------------------------
    ------------------
print()
print(LINE)
print("遥感影像挂接")
print(LINE)
doc = fitz.open()
pg = doc.new_page(width=320, height=200)
pg.insert_text((20, 60), "RS image", fontname="china-s",
    fontsize=16)
png = doc[0].get_pixmap(dpi=100).tobytes("png")
doc.close()
r = c.post("/project/patch/%d/image" % patch_id,
           data={"image": (io.BytesIO(png), "影像.png"),
```

```
                    "image_note": "2025年6月无人机正射影像"},
                content_type="multipart/form-data",
        follow_redirects=True)
body = r.get_data(as_text=True)
ok("已挂接" in body and "<img" in body, "影像上传并显示")
with app.app_context():
        rel = db.session.get(MinePatch, patch_id).image_path
ok(bool(rel), "数据库记录影像路径", rel)
abs_path = os.path.join(TestConfig.UPLOAD_FOLDER,
        rel.replace("uploads/", "", 1))
ok(os.path.isfile(abs_path), "影像文件落盘")
ok(c.get("/static/" + rel).status_code == 200, "影像可经 URL
    访问")
r = c.post("/project/patch/%d/image" % patch_id,
                data={"image": (io.BytesIO(b"x"), "bad.txt")},
                content_type="multipart/form-data",
        follow_redirects=True)
ok("不支持" in r.get_data(as_text=True), "拒绝非图片格式")
c.post("/project/patch/%d/image/delete" % patch_id,
        follow_redirects=True)
with app.app_context():
        ok(db.session.get(MinePatch, patch_id).image_path is
        None, "影像移除后字段清空")
ok(not os.path.isfile(abs_path), "影像文件已删除")
# ------------------------------------------------------------
    ------------------
print()
print(LINE)
print("指标定义维护与多专家矩阵")
print(LINE)
r = c.get("/ahp/indicators")
body = r.get_data(as_text=True)
with app.app_context():
        n_ind = Indicator.query.count()
ok(r.status_code == 200 and n_ind > 0, "指标定义已入库并可查看", "%d
    项" % n_ind)
ok(all(k in body for k in ("一级指标", "二级指标", "三级指标")),
    "指标树按层级展示")
ok("岩质边坡" in body and "平台" in body, "显示各指标启用的微地质单元")
c.post("/ahp/indicators/coverage/edit",
        data={"name": "植被覆盖度", "unit": "%", "ideal_type":
        "benefit",
                "ideal_value": "75", "enabled_units": ["岩质边坡"],
                "description": "自检修改"})
with app.app_context():
        ok(Indicator.query.filter_by(code="coverage").first().de
        scription == "自检修改",
            "指标定义可编辑")
c.post("/ahp/indicators/reset")
with app.app_context():
```

```
    ok("自检修改" not in (Indicator.query.filter_by(code="covera
    ge").first().description or ""),
        "可按系统默认值重置")
base = {"m_0_1": "1.4194", "m_0_2": "1.76", "m_1_2": "1.24"}
for name in ("张工", "李工", "王工"):
    d = dict(base); d.update({"action": "add", "unit_type":
    "碎石边坡",
                              "level": "1", "expert_name":
    name})
    c.post("/ahp/experts", data=d)
with app.app_context():
    ok(ExpertMatrix.query.filter_by(unit_type="碎石边坡",
    level=1).count() == 3,
        "录入 3 位专家的判断矩阵")
r = c.get("/ahp/experts/aggregate?unit_type=碎石边坡&level=1")
body = r.get_data(as_text=True)
ok("汇总后权重" in body and "各专家矩阵的一致性" in body,
   "多专家矩阵几何平均汇总可用")
ws = re.search(r"<th scope=\"row\">权重</th>\s*(.*?)</tr>",
    body, re.S)
vals = re.findall(r">([0-9.]+)</td>", ws.group(1)) if ws
    else []
ok(len(vals) == 3 and abs(sum(map(float, vals)) - 1.0) <
    0.01,
   "汇总权重归一", "、".join(vals))
ok(abs(float(vals[0]) - 0.44) < 0.02, "汇总权重接近论文表 3 取值",
    vals[0] if vals else "")
# ------------------------------------------------------------
    ----------------
print()
print(LINE)
print("批量导入")
print(LINE)
r = c.get("/indicator/import/template")
tpl = r.data.decode("utf-8-sig")
ok(r.status_code == 200 and "单元类型" in tpl, "导入模板可下载")
ok("土石比" in tpl and "植被覆盖度" in tpl, "模板包含指标列")
csv_text = (
    "单元类型,土石比,土层厚度,pH,有机质,Hg,As,Ni,Cu,Cd,Pb,植被覆盖度,Shannon,坡度
    ,坡向\n"
    "碎石边坡,0.34,10,9.24,4.89,0.05,5.17,12.60,11.70,0.04,6.50,
    87.92,0.60,28.75,阳坡\n"
    "平台,0.56,30,8.72,10.92,0.02,3.13,12.33,11.14,0.09,9.98,5
    8.42,0.72,,,\n"
    "不存在单元,1,1,7,1,,,,,,,,,,\n"
)
r = c.post("/indicator/import",
           data={"patch_id": str(patch_id),
                 "datafile": (io.BytesIO(csv_text.encode("ut
    f-8")), "d.csv")},
```

```
                content_type="multipart/form-data",                    12457
        follow_redirects=True)                                         12458
body = r.get_data(as_text=True)                                        12459
ok("导入结果" in body, "导入结果页展示")                                12460
ok("不存在单元" in body, "错误行被报告")                                12461
with app.app_context():                                                12462
        u = MicroUnit.query.filter_by(patch_id=patch_id,               12463
        unit_type="碎石边坡").first()                                   12464
        ind = IndicatorData.query.filter_by(unit_id=u.id).first(12465
        )                                                              12466
        ok(abs(ind.soil_ratio - 0.34) < 1e-6 and                       12467
        abs(ind.coverage - 87.92) < 1e-6,                              12468
            "导入数据已写入数据库")                                     12469
r = c.post("/indicator/import",                                        12470
                data={"patch_id": str(patch_id),                       12471
                      "datafile": (io.BytesIO(b"x"), "d.pdf")},        12472
                content_type="multipart/form-data",                    12473
        follow_redirects=True)                                         12474
ok("仅支持" in r.get_data(as_text=True), "拒绝不支持的导入格式")        12475
# ------------------------------------------------------------------12476
        ------------------                                             12477
print()                                                                12478
print(LINE)                                                            12479
print("图表与 PDF 导出")                                               12480
print(LINE)                                                            12481
svg = charts.hbars([("岩质边坡", 0.54, "0.5401"), ("平台", 0.84,       12482
        "0.8376")])                                                    12483
ok("<svg" in svg and "0.8376" in svg, "条形图生成且带数值标签")        12484
svg2 = charts.donut([("植被指标", 0.6, "0.6000"), ("土壤指标", 0.4,    12485
        "0.4000")])                                                    12486
ok("<svg" in svg2 and "占比" in svg2, "环形图生成且带图例")             12487
#  报告是评价结论的呈现，需先真实执行一次评价才能打开                  12488
with app.app_context():                                                12489
        from app.views.evaluate import load_patch_input,               12490
        save_evaluation                                                12491
        save_evaluation(patch_id, load_patch_input(patch_id))          12492
r = c.get("/report/%d" % patch_id)                                     12493
body = r.get_data(as_text=True)                                        12494
ok(body.count("<svg") >= 2, "得分明细页含图表", "%d 个 SVG" %          12495
        body.count("<svg"))                                            12496
exe = browser.browser_path()                                           12497
if exe is None:                                                        12498
        print("  [跳过] 未找到浏览器，PDF 导出无法验证")               12499
else:                                                                  12500
        r = c.get("/report/%d/pdf" % patch_id)                         12501
        data = r.data                                                  12502
        ok(data[:4] == b"%PDF", "PDF 报告导出成功", "%d 字节" %        12503
        len(data))                                                     12504
        d = fitz.open(stream=data, filetype="pdf")                     12505
        txt = "".join(d[i].get_text() for i in                         12506
```

```
    range(d.page_count))                                                    12507
    ok("自然恢复适宜性评价报告" in txt, "PDF 含报告标题")                    12508
    ok("评价结论" in txt and "得分明细" in txt, "PDF 含结论与明细章节")      12509
    ok(d.page_count >= 2, "PDF 页数合理", "%d 页" % d.page_count)           12510
    d.close()                                                               12511
print()                                                                     12512
print(LINE)                                                                 12513
print("补齐模块自检全部通过 √")                                            12514
print(LINE)                                                                 12515
#  ==== 测试/test_workflow.py ====                                         12516
#  -*- coding: utf-8 -*-                                                   12517
"""流程门禁自检：前置步骤未完成时，后续模块必须拦得住。                    12518
论文 4.1 节确定的评价流程存在先后关系，缺任何一环都不应产出评价结论。      12519
本测试逐个构造「缺前置」的场景，验证系统不会放行。                        12520
"""                                                                         12521
import os                                                                   12522
import sys                                                                  12523
sys.stdout.reconfigure(encoding="utf-8")                                    12524
#  本脚本位于 测试/ 子目录，先把仓库根目录加入模块搜索路径，                12525
#  使 `from app import ...` 与 app/app.db 的定位不受影响。                 12526
ROOT = os.path.dirname(os.path.dirname(os.path.abspath(__fil                12527
    e__)))                                                                  12528
if ROOT not in sys.path:                                                    12529
    sys.path.insert(0, ROOT)                                                12530
from app import create_app                                                  12531
from app.models import (db, Project, MinePatch, MicroUnit,                  12532
    IndicatorData,                                                          12533
                        ConstraintCheck, EvaluationResult)                  12534
from app.services import workflow as wf                                     12535
LINE = "=" * 72                                                             12536
BASE = ROOT                                                                 12537
class TestConfig:                                                           12538
    SECRET_KEY = "test"                                                     12539
    SQLALCHEMY_DATABASE_URI = "sqlite:///" +                                12540
    os.path.join(BASE, "app", "app.db")                                     12541
    SQLALCHEMY_TRACK_MODIFICATIONS = False                                  12542
    WTF_CSRF_ENABLED = False                                                12543
    TESTING = True                                                          12544
app = create_app(TestConfig)                                                12545
c = app.test_client()                                                       12546
c.post("/login", data={"username": "admin", "password":                     12547
    "admin123"})                                                            12548
_fails = []                                                                 12549
def ok(flag, label, extra=""):                                              12550
    print("  %s %-46s %s" % ("[符合]" if flag else "[不符合]",              12551
    label, extra))                                                          12552
    if not flag:                                                            12553
        _fails.append(label)                                                12554
def purge_fixtures():                                                       12555
    """开跑前先清掉上一次可能残留的自检项目。                              12556
```

```
    自检项目按 code 前缀命名（WF-）；如果上一次跑到一半被打断（报错、被中断），
    清理函数没执行到，残留的项目就会出现在工作台与项目列表里 ——
    生成软著截图时这些测试数据会被拍进去。
    """
    with app.app_context():
        leftovers = Project.query.filter(Project.code.like("
    WF-%")).all()
        for p in leftovers:
            for m in p.patches.all():
                db.session.delete(m)
            db.session.delete(p)
        db.session.commit()
        return len(leftovers)
def new_patch(tag):
    with app.app_context():
        p = Project(name="门禁-" + tag, code="WF-" + tag)
        db.session.add(p); db.session.flush()
        patch = MinePatch(project_id=p.id, name="门禁图斑-" +
    tag)
        db.session.add(patch); db.session.flush()
        pid = patch.id
        db.session.commit()
    return pid
def add_units(pid):
    with app.app_context():
        db.session.add(MicroUnit(patch_id=pid,
    unit_type="碎石边坡",
                                 area=1.0, ratio=1.0))
        db.session.commit()
def add_constraint(pid, passed=True):
    with app.app_context():
        db.session.add(ConstraintCheck(
            patch_id=pid, policy_pass=True,
    location_pass=passed,
            pollution_pass=True, geo_pass=True,
    farmland_pass=True,
            passed=passed, reason="全部通过" if passed else
    "未通过：区位约束"))
        db.session.commit()
def add_data(pid):
    with app.app_context():
        u = MicroUnit.query.filter_by(patch_id=pid).first()
        db.session.add(IndicatorData(unit_id=u.id,
    soil_ratio=0.34, thickness=10,
                                     ph=9.24,
    organic_matter=4.89, coverage=87.92,
                                     shannon=0.60,
    slope=28.75, aspect="阳坡"))
        db.session.commit()
def cleanup(pid, tag):
```

```
    with app.app_context():
        from app.models import ScoreDetail
        # 先删得分明细，再删评价记录，避免留下孤儿明细
        rids = [r.id for r in EvaluationResult.query.filter_by(patch_id=pid)]
        if rids:
            ScoreDetail.query.filter(ScoreDetail.result_id.in_(rids)).delete(
                synchronize_session=False)
        EvaluationResult.query.filter_by(patch_id=pid).delete()
        ConstraintCheck.query.filter_by(patch_id=pid).delete()
        uids = [u.id for u in MicroUnit.query.filter_by(patch_id=pid)]
        if uids:
            IndicatorData.query.filter(IndicatorData.unit_id.in_(uids)).delete(
                synchronize_session=False)
        MicroUnit.query.filter_by(patch_id=pid).delete()
        m = db.session.get(MinePatch, pid)
        if m:
            db.session.delete(m)
        pr = Project.query.filter_by(name="门禁-" + tag).first()
        if pr:
            db.session.delete(pr)
        db.session.commit()
def results_of(pid):
    with app.app_context():
        return EvaluationResult.query.filter_by(patch_id=pid).count()
def post_eval(pid):
    return c.post("/evaluate/%d" % pid, data={},
    follow_redirects=False)
def get_report(pid):
    return c.get("/report/%d" % pid, follow_redirects=False)
# ------------------------------------------------------------------
print(LINE)
print("一、前置条件缺失时，评价必须被拦住")
print(LINE)
print("  （开跑前清掉 %d 个上一次残留的自检项目）" % purge_fixtures())
print()
print("场景 1：连微地质单元都没划分")
pid = new_patch("1")
r = post_eval(pid)
n = results_of(pid)
ok(r.status_code == 302 and n == 0, "未划分单元 → 不产生评价结果",
   "HTTP %d，结果数 %d" % (r.status_code, n))
```

```
with app.app_context():
    pr = wf.patch_progress(pid)
steps = [b["step"] for b in pr["blocking"]]
ok(not pr["ready"] and "units" in steps,
   "拦原因包含「划分微地质单元」", "、".join(steps))
ok(steps == ["constraint", "units"],
   "拦截顺序符合论文 4.1 节：限制性判定在前，划分单元在后",
   "、".join(steps))
cleanup(pid, "1")
print()
print("场景 2：有单元，但未做限制性条件判定")
pid = new_patch("2"); add_units(pid); add_data(pid)
r = post_eval(pid)
n = results_of(pid)
ok(r.status_code == 302 and n == 0, "未做限制性判定 → 不产生评价结果",
   "结果数 %d" % n)
with app.app_context():
    pr = wf.patch_progress(pid)
ok(any(b["step"] == "constraint" for b in pr["blocking"]),
   "拦原因指向「执行限制性条件判定」")
cleanup(pid, "2")
print()
print("场景 3：限制性条件判定【未通过】")
pid = new_patch("3"); add_units(pid); add_data(pid);
    add_constraint(pid, passed=False)
r = post_eval(pid)
n = results_of(pid)
ok(r.status_code == 302 and n == 0, "判定未通过 → 不进入定量评价",
   "结果数 %d" % n)
with app.app_context():
    pr = wf.patch_progress(pid)
ok(not pr["constraint"]["passed"], "流程状态显示判定未通过")
blocked = [b for b in pr["blocking"] if b["step"] ==
    "constraint"]
# 提示只写「缺什么」：label 说明未通过，reason 直接引用判定记录里没过的项
ok(bool(blocked) and "未通过" in blocked[0]["label"],
   "拦提示指明限制性条件判定未通过", blocked[0]["label"] if blocked else
    "")
ok(bool(blocked) and "区位约束" in (blocked[0]["reason"] or ""),
   "并把具体没通过的条件带出来", blocked[0]["reason"] if blocked else "")
cleanup(pid, "3")
print()
print("场景 4：判定通过，但指标数据全空")
pid = new_patch("4"); add_units(pid); add_constraint(pid,
    passed=True)
r = post_eval(pid)
n = results_of(pid)
ok(r.status_code == 302 and n == 0, "数据未录入 → 不产生评价结果",
   "结果数 %d" % n)
with app.app_context():
```

```
    pr = wf.patch_progress(pid)
ok(any(b["step"] == "data" for b in pr["blocking"]),
   "拦原因指向「录入指标数据」")
ok(pr["data"]["missing"] == ["碎石边坡"],
   "指出具体缺哪个单元的数据", "、".join(pr["data"]["missing"]))
cleanup(pid, "4")
print()
print("场景 5：单元面积占比合计不为 100%")
pid = new_patch("5"); add_constraint(pid, passed=True)
with app.app_context():
    u = MicroUnit(patch_id=pid, unit_type="碎石边坡", area=1.0,
    ratio=0.5)
    db.session.add(u); db.session.commit()
    db.session.add(IndicatorData(unit_id=u.id,
    soil_ratio=0.3, thickness=10,
                                 ph=8.0,
    organic_matter=5.0, coverage=80.0,
                                 shannon=0.6, slope=20.0,
    aspect="阳坡"))
    db.session.commit()
r = post_eval(pid)
ok(results_of(pid) == 0, "占比不足 100% → 不产生评价结果")
cleanup(pid, "5")
print()
print(LINE)
print("二、前置条件齐备时才放行")
print(LINE)
pid = new_patch("OK"); add_units(pid); add_constraint(pid,
    passed=True); add_data(pid)
with app.app_context():
    pr = wf.patch_progress(pid)
ok(pr["ready"], "单元 + 判定通过 + 数据齐备 → 判定为可执行")
r = post_eval(pid)
n = results_of(pid)
ok(n == 1, "此时执行评价成功产生结果", "结果数 %d" % n)
print("  重定向到:", r.headers.get("Location"))
print()
print(LINE)
print("三、报告不得先于评价产生")
print(LINE)
pid2 = new_patch("R")    # 什么都不做
r = get_report(pid2)
ok(r.status_code == 302 and "/report/" ==
    (r.headers.get("Location") or ""),
   "前置缺失时 /report/<id> 被退回列表", "HTTP %d" % r.status_code)
for suffix in ("/pdf", "/word"):
    rr = c.get("/report/%d%s" % (pid2, suffix),
    follow_redirects=False)
    ok(rr.status_code == 302, "导出 %s 同样被拦住" % suffix, "HTTP
    %d" % rr.status_code)
```

```
cleanup(pid2, "R")
cleanup(pid, "OK")
print()
print(LINE)
print("四、前置齐备但从未执行评价时，报告仍不得产生")
print(LINE)
#  报告是评价结论的呈现：即使前置步骤都做完了，只要没真正执行过定量评价，
#  也不应凭空推算出结论。
pid3 = new_patch("N")
add_units(pid3); add_constraint(pid3, passed=True);
    add_data(pid3)
with app.app_context():
    pr = wf.report_progress(pid3)
ok(not pr["ready"], "无评价记录 → 报告流程判定为未就绪")
ok(bool(pr["blocking"]) and pr["blocking"][0]["step"] ==
    "evaluate",
   "拦原因指向「执行适宜性评价」")
r = get_report(pid3)
ok(r.status_code == 302 and results_of(pid3) == 0,
   "直接访问 /report/<id> 被退回，且不产生任何评价结果",
   "HTTP %d" % r.status_code)
#  真正执行评价之后，报告才放行，且取值与评价记录一致
r = post_eval(pid3)
ok(results_of(pid3) == 1, "先执行评价 → 产生评价记录")
with app.app_context():
    pr = wf.report_progress(pid3)
    rec = pr["result"]
ok(pr["ready"] and rec is not None, "有评价记录后，报告流程就绪")
rr = get_report(pid3)
body = rr.get_data(as_text=True)
ok(rr.status_code == 200, "此时可打开得分明细页", "HTTP %d" %
    rr.status_code)
ok(("%.4f" % rec.final_score) in body,
   "报告数值取自评价记录，而非重新计算",
   "F=%.4f" % rec.final_score)
cleanup(pid3, "N")
print()
print(LINE)
print("五、内置演示数据的清理不得波及用户自建数据")
print(LINE)
#  回归保护：seed_demo.clean_case() 曾执行 Project.query.delete()，
#  导致用户新建的项目在每次启动时被连带删除。
import seed_demo as sd
with app.app_context():
    keep_p = Project(name="用户自建项目-守护", code="KEEP-001")
    db.session.add(keep_p); db.session.flush()
    keep_patch = MinePatch(project_id=keep_p.id,
    name="用户自建图斑-守护")
    db.session.add(keep_patch); db.session.flush()
    keep_pid = keep_p.id
```

```
        keep_patch_id = keep_patch.id                                    12807
        db.session.commit()                                              12808
with app.app_context():                                                  12809
        sd.clean_case(reset=False)                                       12810
with app.app_context():                                                  12811
        still_p = db.session.get(Project, keep_pid)                      12812
        still_patch = db.session.get(MinePatch, keep_patch_id)           12813
ok(still_p is not None, "清理演示数据后，用户自建项目仍存在")                12814
ok(still_patch is not None, "清理演示数据后，用户自建图斑仍存在")            12815
#  演示数据清掉后应能重新写入，保证演示环境可恢复                           12816
sd.seed()                                                                12817
with app.app_context():                                                  12818
        demo_back = Project.query.filter_by(code=sd.DEMO_CODE).f         12819
        irst()                                                           12820
ok(demo_back is not None, "演示案例可重新写入（幂等恢复）")                  12821
with app.app_context():                                                  12822
        m = db.session.get(MinePatch, keep_patch_id)                     12823
        if m:                                                            12824
                db.session.delete(m)                                     12825
        p = db.session.get(Project, keep_pid)                            12826
        if p:                                                            12827
                db.session.delete(p)                                     12828
        db.session.commit()                                              12829
print()                                                                  12830
print(LINE)                                                              12831
print("六、同一图斑不得重复划分同类型微地质单元")                          12832
print(LINE)                                                              12833
#  论文 4.1 节把图斑分为平台、碎石边坡、岩质边坡三类，每类至多一个。        12834
#  允许重复会让面积占比、按类型组织的指标数据与批量导入对不上号。          12835
pid4 = new_patch("D")                                                    12836
c.post("/unit/%d/add" % pid4, data={"unit_type": "平台",                 12837
        "area": "1.0"},                                                  12838
           follow_redirects=True)                                        12839
with app.app_context():                                                  12840
        n1 = MicroUnit.query.filter_by(patch_id=pid4).count()            12841
ok(n1 == 1, "首次添加「平台」成功", "单元数 %d" % n1)                      12842
r = c.post("/unit/%d/add" % pid4, data={"unit_type": "平台",             12843
        "area": "2.0"},                                                  12844
                follow_redirects=True)                                   12845
with app.app_context():                                                  12846
        n2 = MicroUnit.query.filter_by(patch_id=pid4).count()            12847
ok(n2 == 1, "重复添加同类型被拒绝", "单元数 %d" % n2)                      12848
ok("同一类型只能划分一个" in r.get_data(as_text=True), "给出重复提示")      12849
r = c.post("/unit/%d/add" % pid4, data={"unit_type": "火山口",           12850
        "area": "1.0"},                                                  12851
                follow_redirects=True)                                   12852
with app.app_context():                                                  12853
        n3 = MicroUnit.query.filter_by(patch_id=pid4).count()            12854
ok(n3 == 1, "非法单元类型被拒绝", "单元数 %d" % n3)                        12855
c.post("/unit/%d/add" % pid4, data={"unit_type": "碎石边坡",             12856
```

```
        "area": "3.0"},                                                  12857
            follow_redirects=True)                                       12858
with app.app_context():                                                  12859
        uid = MicroUnit.query.filter_by(patch_id=pid4,                   12860
                                        unit_type="碎石边坡").first()    12861
        ).id                                                             12862
r = c.post("/unit/edit/%d" % uid,                                        12863
                   data={"unit_type": "平台", "area": "3.0"},            12864
        follow_redirects=True)                                           12865
with app.app_context():                                                  12866
        types = sorted(u.unit_type for u in                              12867
                       MicroUnit.query.filter_by(patch_id=pid4)          12868
        all())                                                           12869
ok(types == ["平台", "碎石边坡"], "改类型时同样拦截重复", "、".join(types))  12870
cleanup(pid4, "D")                                                       12871
print()                                                                  12872
print(LINE)                                                              12873
if _fails:                                                               12874
        print("存在未通过项：")                                          12875
        for f in _fails:                                                 12876
            print("  -", f)                                             12877
        sys.exit(1)                                                      12878
print("流程门禁自检全部通过 √")                                         12879
print(LINE)                                                              12880
#  ==== 测试/test_weights.py ====                                        12881
#  -*- coding: utf-8 -*-                                                 12882
"""权重生效链路自检：AHP 算出的权重必须真正参与评价计算。                12883
此前 weight_schemes / weight_items 两张表从未被写入，评价引擎也只用      12884
代码里的论文表 3 默认权重，相当于权重模块算完即弃。本测试守住这条链路：  12885
    默认 → 论文表 3                                                      12886
    保存 AHP 权重 → 评价改用新权重，结果随之变化                         12887
    恢复默认 → 结果回到论文取值                                          12888
"""                                                                      12889
import os                                                                12890
import sys                                                               12891
sys.stdout.reconfigure(encoding="utf-8")                                 12892
#  本脚本位于 测试/ 子目录，先把仓库根目录加入模块搜索路径，             12893
#  使 `from app import ...` 与 app/app.db 的定位不受影响。               12894
ROOT = os.path.dirname(os.path.dirname(os.path.abspath(__fil             12895
        e__)))                                                           12896
if ROOT not in sys.path:                                                 12897
        sys.path.insert(0, ROOT)                                         12898
from app import create_app                                               12899
from app.models import (db, Project, MinePatch, MicroUnit,               12900
        IndicatorData,                                                   12901
                                ConstraintCheck, EvaluationResult,       12902
        WeightScheme,                                                    12903
                                WeightItem)                              12904
from app.services import workflow as wf                                  12905
from app.services import weight_service as ws                            12906
```

```
LINE = "=" * 72
BASE = ROOT
class TestConfig:
    SECRET_KEY = "test"
    SQLALCHEMY_DATABASE_URI = "sqlite:///" +
    os.path.join(BASE, "app", "app.db")
    SQLALCHEMY_TRACK_MODIFICATIONS = False
    WTF_CSRF_ENABLED = False
    TESTING = True
app = create_app(TestConfig)
c = app.test_client()
c.post("/login", data={"username": "admin", "password":
    "admin123"})
_fails = []
def ok(flag, label, extra=""):
    print("  %s %-46s %s" % ("[符合]" if flag else "[不符合]",
    label, extra))
    if not flag:
        _fails.append(label)
TAG = "W"
UNIT_TYPE = "碎石边坡"
def build_case():
    """建一个只含单一微地质单元的图斑，便于观察权重变化对得分的影响。"""
    with app.app_context():
        p = Project(name="权重测试-%s" % TAG, code="WEIGHT-%s"
    % TAG)
        db.session.add(p); db.session.flush()
        patch = MinePatch(project_id=p.id, name="权重测试图斑-%s"
    % TAG)
        db.session.add(patch); db.session.flush()
        db.session.add(MicroUnit(patch_id=patch.id,
    unit_type=UNIT_TYPE,
                                 area=1.0, ratio=1.0))
        db.session.flush()
        u = MicroUnit.query.filter_by(patch_id=patch.id).fir
    st()
        db.session.add(IndicatorData(
            unit_id=u.id, soil_ratio=0.34, thickness=10,
    ph=9.24,
            organic_matter=4.89, coverage=87.92,
    shannon=0.60,
            slope=28.75, aspect="阳坡"))
        db.session.add(ConstraintCheck(
            patch_id=patch.id, policy_pass=True,
    location_pass=True,
            pollution_pass=True, geo_pass=True,
    farmland_pass=True,
            passed=True, reason="全部通过"))
        db.session.commit()
        return patch.id
```

```
def purge_fixtures():
    """开跑前清掉上一次可能残留的自检项目（code 前缀 WEIGHT-）。
    上一次跑到一半被打断时清理函数执行不到，残留项目会出现在工作台与
    项目列表里，生成软著截图时会被拍进去。
    """
    with app.app_context():
        from app.models import Project, ReferenceEcosystem
        leftovers = Project.query.filter(Project.code.like("WEIGHT-%")).all()
        for p in leftovers:
            for m in p.patches.all():
                uids = [u.id for u in MicroUnit.query.filter_by(patch_id=m.id)]
                if uids:
                    IndicatorData.query.filter(
                        IndicatorData.unit_id.in_(uids)).delete(
                        synchronize_session=False)
                EvaluationResult.query.filter_by(patch_id=m.id).delete()
                ConstraintCheck.query.filter_by(patch_id=m.id).delete()
                ReferenceEcosystem.query.filter_by(patch_id=m.id).delete()
                MicroUnit.query.filter_by(patch_id=m.id).delete()
                db.session.delete(m)
            db.session.delete(p)
        db.session.commit()
        return len(leftovers)
def cleanup(pid):
    with app.app_context():
        from app.models import ScoreDetail
        rids = [r.id for r in EvaluationResult.query.filter_by(patch_id=pid)]
        if rids:
            ScoreDetail.query.filter(ScoreDetail.result_id.in_(rids)).delete(
                synchronize_session=False)
        EvaluationResult.query.filter_by(patch_id=pid).delete()
        ConstraintCheck.query.filter_by(patch_id=pid).delete()
        uids = [u.id for u in MicroUnit.query.filter_by(patch_id=pid)]
        if uids:
            IndicatorData.query.filter(IndicatorData.unit_id.in_(uids)).delete(
                synchronize_session=False)
        MicroUnit.query.filter_by(patch_id=pid).delete()
```

```
            m = db.session.get(MinePatch, pid)
            if m:
                db.session.delete(m)
            p = Project.query.filter_by(code="WEIGHT-%s" %
        TAG).first()
            if p:
                db.session.delete(p)
            db.session.commit()
    def run_eval(pid):
        c.post("/evaluate/%d" % pid, data={},
        follow_redirects=False)
        with app.app_context():
            return EvaluationResult.query.filter_by(patch_id=pid
        ) \
                .order_by(EvaluationResult.id.desc()).first().fi
        nal_score
    print(LINE)
    print("一、默认应使用系统内置权重")
    print(LINE)
    print("  （开跑前清掉 %d 个上一次残留的自检项目）" % purge_fixtures())
    with app.app_context():
        ws.reset_weights(UNIT_TYPE)
        w, label, scheme = ws.effective_weights(UNIT_TYPE)
    ok(scheme is None and "系统默认权重" in label,
       "未配置时权重来源为系统默认权重", label)
    ok(abs(w["l1"]["土壤指标"] - 0.44) < 1e-9, "一级权重取自系统内置权重",
       "土壤指标 %.2f" % w["l1"]["土壤指标"])
    pid = build_case()
    base_score = run_eval(pid)
    print("  默认权重下的评价值 F = %.4f" % base_score)
    print()
    print(LINE)
    print("二、保存 AHP 算出的权重后，评价应改用新权重")
    print(LINE)
    with app.app_context():
        # 模拟 AHP 计算结果：把土壤指标权重提高、植被与地形降低（仍和为 1）
        ws.save_weights(UNIT_TYPE, [("soil", 0.70), ("veg",
        0.20),
                                    ("terrain", 0.10)])
        n_items = WeightItem.query.count()
    ok(n_items >= 3, "weight_items 表已写入权重条目", "%d 条" % n_items)
    with app.app_context():
        w2, label2, scheme2 = ws.effective_weights(UNIT_TYPE)
    ok(scheme2 is not None and "自定义" in label2, "权重来源变为自定义方案",
        label2)
    ok(abs(w2["l1"]["土壤指标"] - 0.70) < 1e-9, "新权重已生效",
       "土壤指标 %.2f" % w2["l1"]["土壤指标"])
    ok(abs(w2["l1"]["植被指标"] - 0.20) < 1e-9, "同批保存的其他项也已生效")
    ok(abs(w2["l2"]["土壤指标"]["土石比"] - 0.25) < 1e-9,
       "未覆盖的二级权重仍保留系统内置取值",
```

```
    "土石比 %.2f" % w2["l2"]["土壤指标"]["土石比"])
new_score = run_eval(pid)
print("   新权重下的评价值 F = %.4f" % new_score)
ok(abs(new_score - base_score) > 1e-4,
    "权重变化确实改变了评价结果",
    "%.4f → %.4f" % (base_score, new_score))
print()
print(LINE)
print("三、结果应记录所用权重来源")
print(LINE)
import json
with app.app_context():
    rec = EvaluationResult.query.filter_by(patch_id=pid) \
        .order_by(EvaluationResult.id.desc()).first()
    stored = json.loads(rec.unit_scores)
ok(any(u.get("weight_source") for u in stored),
    "评价记录中标注了权重来源",
    stored[0].get("weight_source") or "")
print()
print(LINE)
print("四、恢复默认后应回到系统默认权重")
print(LINE)
with app.app_context():
    ws.reset_weights(UNIT_TYPE)
    w3, label3, scheme3 = ws.effective_weights(UNIT_TYPE)
ok(scheme3 is None and "系统默认权重" in label3,
    "已恢复为系统默认权重", label3)
back_score = run_eval(pid)
print("   恢复后评价值 F = %.4f（默认时 %.4f）" % (back_score,
    base_score))
ok(abs(back_score - base_score) < 1e-9, "评价结果与默认时一致")
print()
print(LINE)
print("五、权重首页概览")
print(LINE)
with app.app_context():
    rows = ws.summary()
ok(len(rows) == 3, "三种微地质单元均有权重来源说明", "%d 行" % len(rows))
r = c.get("/ahp/")
body = r.get_data(as_text=True)
ok(r.status_code == 200 and "当前生效权重" in body, "权重首页正常展示")
print()
print(LINE)
print("六、一致性不通过的矩阵不得保存生效")
print(LINE)
with app.app_context():
    ws.reset_weights(UNIT_TYPE)
n = 3
# 明显矛盾的矩阵：a12=9, a13=1/9, a23=9 —— CR 必然超限
mats = [(0, 1, "9"), (0, 2, "0.1111"), (1, 2, "9")]
```

```
data = {"csrf_token": "", "action": "save"}
for i, j, v in mats:
    data["m_%d_%d" % (i, j)] = v
r = c.post("/ahp/matrix?unit_type=%s&level=1" % UNIT_TYPE,
    data=data,
              follow_redirects=True)
with app.app_context():
    _, label4, scheme4 = ws.effective_weights(UNIT_TYPE)
ok(scheme4 is None, "CR 超限的矩阵未被保存生效", label4)
ok("一致性" in r.get_data(as_text=True), "页面给出了未通过一致性的提示")
cleanup(pid)
print()
print(LINE)
if _fails:
    print("存在未通过项：")
    for f in _fails:
        print("  -", f)
    sys.exit(1)
print("权重生效链路自检全部通过 √")
print(LINE)
#  ==== app/services/__init__.py ====
#  -*- coding: utf-8 -*-
#  ==== app/services/cleanup.py ====
#  -*- coding: utf-8 -*-
"""级联清理：删图斑、删项目时，把挂在它下面的数据一并清掉。
为什么单独抽出来：`Project.patches` 与 `MinePatch.units` 上有 ORM 级联，
但图斑下面的**指标数据、限制性判定、评价结果、得分明细、参照生态系统**
都没有级联关系。原先只有「删除图斑」那条路径里手写了清理，
于是「删除项目」会留下一批孤儿行——自检实测：删一个项目后，
上面五张表各残留 1 行（项目与图斑本身都删掉了，数据却还在库里）。
两条删除路径都调用这里，才不会漏。
"""
from app.models import (db, MinePatch, MicroUnit,
    IndicatorData,
                        EvaluationResult, ScoreDetail,
    ConstraintCheck,
                        ReferenceEcosystem)
def delete_patch_children(patch_id):
    """清掉挂在某个图斑下的全部附属数据（不含图斑本身）。
    返回各表清理条数，便于调用方打印或断言。
    """
    counts = {}
    unit_ids = [u.id for u in MicroUnit.query.filter_by(patch_id=patch_id).all()]
    if unit_ids:
        counts["indicator_data"] = IndicatorData.query.filter(
            IndicatorData.unit_id.in_(unit_ids)).delete(synchronize_session=False)
    result_ids = [r.id for r in
```

```
                EvaluationResult.query.filter_by(patch_id=
    patch_id).all()]
    if result_ids:
        counts["score_details"] = ScoreDetail.query.filter(
            ScoreDetail.result_id.in_(result_ids)).delete(sy
    nchronize_session=False)
    counts["evaluation_results"] = EvaluationResult.query.fi
    lter_by(
        patch_id=patch_id).delete(synchronize_session=False)
    counts["constraint_checks"] = ConstraintCheck.query.filt
    er_by(
        patch_id=patch_id).delete(synchronize_session=False)
    counts["micro_units"] = MicroUnit.query.filter_by(
        patch_id=patch_id).delete(synchronize_session=False)
    counts["reference_ecosystems"] = ReferenceEcosystem.quer
    y.filter_by(
        patch_id=patch_id).delete(synchronize_session=False)
    return counts
def delete_patch(patch_id):
    """删除一个图斑及其全部附属数据。图斑不存在时返回 （None, {}）。"""
    patch = db.session.get(MinePatch, patch_id)
    if patch is None:
        return None, {}
    counts = delete_patch_children(patch_id)
    db.session.delete(patch)
    db.session.commit()
    return patch, counts
def delete_project(project):
    """删除一个项目：先清掉每个图斑的附属数据，再删项目本身。
    项目下的图斑由 `Project.patches` 的 ORM 级联删除，但图斑下面那五张表
    必须先手工清，否则会留孤儿。
    """
    cleaned = 0
    for patch in project.patches.all():
        counts = delete_patch_children(patch.id)
        cleaned += sum(counts.values())
    name = project.name
    db.session.delete(project)
    db.session.commit()
    return name, cleaned
# ==== app/services/standard_check.py ====
# -*- coding: utf-8 -*-
"""评价结论与地方标准 / 政策文件的对照。
标准知识库只作文件储备与比对，**不参与评价因子 F 的计算**：评价仍按内置的
指标体系与权重算，算完之后再拿结论去对照文件里的判定项，回答
「这个结论是否符合当地文件的要求」。
对照结果有三种，映射不上的项如实标为「待人工判断」，不由系统替文件下结论：
    符合          —— 系统里有对应数据，且满足文件里的判定条件
    不符合        —— 系统里有对应数据，但不满足
    待人工判断    —— 文件里的定性要求，系统内没有对应数据（如「地形地貌与周边协调」）
```

```
对照用到的数据全部来自系统已有记录：限制性条件判定结果、各微地质单元实测值、
参照生态系统基准、最近一次评价记录。
"""
import re
from app.models import (db, MinePatch, MicroUnit,
    IndicatorData, ConstraintCheck,
                           Standard, StandardRule,
    ReferenceEcosystem, EvaluationResult)
OK = "符合"
BAD = "不符合"
MANUAL = "待人工判断"
# 结论 → 页面上的标签样式（沿用既有的 tag 样式，不新增样式）
TAG_CLASS = {OK: "tag--ok", BAD: "tag--warn", MANUAL:
    "tag--muted"}
def province_of(region):
    """从行政区里认出已收录的省份：「河北省唐山市玉田县」→「河北」。
    库里没有这个省份的标准时返回 None，由调用方给出「未收录」的提示。
    """
    if not region:
        return None
    provinces = {p for (p,) in db.session.query(Standard.pro
    vince).distinct()
                 if p}
    for p in sorted(provinces, key=len, reverse=True):
        if p in region:
            return p
    return None
def _num(text):
    if text is None:
        return None
    m = re.search(r"-?\d+(?:\.\d+)?", str(text))
    return float(m.group(0)) if m else None
def _cmp(value, op, threshold):
    """按比较符判断；阈值不是数字或取值缺失时返回 None（无法判断）。"""
    t = _num(threshold)
    if value is None or t is None:
        return None
    try:
        v = float(value)
    except (TypeError, ValueError):
        return None
    if op == ">=":
        return v >= t
    if op == ">":
        return v > t
    if op == "<=":
        return v <= t
    if op == "<":
        return v < t
    if op == "==":
```

```
            return abs(v - t) < 1e-9                                    13257
        return None                                                     13258
def _coverage_stats(patch_id):                                          13259
    """各单元植被覆盖度的面积加权值与最低值（只统计录了实测值的单元）。"""                  13260
    rows = (db.session.query(MicroUnit, IndicatorData)                  13261
            .outerjoin(IndicatorData, IndicatorData.unit_id             13262
    == MicroUnit.id)                                                    13263
            .filter(MicroUnit.patch_id == patch_id).all())              13264
    values, num, den = [], 0.0, 0.0                                     13265
    for unit, data in rows:                                             13266
        cov = getattr(data, "coverage", None) if data is                13267
    not None else None                                                  13268
        if cov is None:                                                 13269
            continue                                                    13270
        values.append(cov)                                              13271
        w = unit.ratio or 0.0                                           13272
        num += cov * w                                                  13273
        den += w                                                        13274
    if not values:                                                      13275
        return None, None                                               13276
    return (num / den if den else None), min(values)                    13277
def _context(patch):                                                    13278
    """把对照要用到的数据一次性取出来。"""                                      13279
    check = (ConstraintCheck.query.filter_by(patch_id=patch             13280
    id)                                                                 13281
             .order_by(ConstraintCheck.id.desc()).first())              13282
    ref = ReferenceEcosystem.query.filter_by(patch_id=patch             13283
    id).first()                                                         13284
    result = (EvaluationResult.query.filter_by(patch_id=pat             13285
    h.id)                                                               13286
              .order_by(EvaluationResult.id.desc()).first())            13287
    weighted, lowest = _coverage_stats(patch.id)                        13288
    return {                                                            13289
        "patch": patch,                                                 13290
        "constraint": check,                                            13291
        "benchmark": getattr(ref, "coverage_benchmark",                 13292
    None) if ref else None,                                             13293
        "has_reference": ref is not None,                               13294
        "result": result,                                               13295
        "grade": getattr(result, "grade", None) if result               13296
    else None,                                                          13297
        "score": getattr(result, "final_score", None) if                13298
    result else None,                                                   13299
        "coverage": weighted,                                           13300
        "coverage_min": lowest,                                         13301
    }                                                                   13302
def _judge(item, op, threshold, ctx):                                   13303
    """把一条文件规则映射到系统数据，返回 （结论， 依据说明）。"""                        13304
    name = item or ""                                                   13305
    cov, cov_min = ctx["coverage"], ctx["coverage_min"]                 13306
```

```
    check = ctx["constraint"]
    def cov_text():
        if cov is None:
            return "尚未录入植被覆盖度"
        return "面积加权覆盖度 %.2f%%（最低单元 %.2f%%）" % (cov,
cov_min)
    # ---- 植被覆盖度（含趋势）----
    if "趋势" in name:
        return MANUAL, "需连续多年的监测数据，系统内没有时间序列"
    if "植被覆盖" in name:
        if cov is None:
            return MANUAL, cov_text()
        r = _cmp(cov, op, threshold)
        if r is None:
            return MANUAL, cov_text() + "；文件给出的条件为「%s %s」"
% (op, threshold)
        return (OK if r else BAD), cov_text()
    # ---- 参照生态系统 ----
    if "参照生态系统调查" in name or "参照系调查" in name:
        return ((OK, "已录入参照生态系统的植被覆盖度与生物多样性基准")
                if ctx["has_reference"] else
                (MANUAL, "尚未录入参照生态系统基准"))
    if "参照系" in name:
        if cov is None or ctx["benchmark"] is None:
            return MANUAL, "缺参照生态系统基准或覆盖度实测值"
        gap = (ctx["benchmark"] - cov)
        gap = 0.0 if gap < 0 else gap
        r = _cmp(gap, op, threshold)
        if r is None:
            return MANUAL, "与参照系差距 %.2f 个百分点" % gap
        return (OK if r else BAD), "与参照系差距 %.2f 个百分点（基准
%.2f%%）" % (
            gap, ctx["benchmark"])
    # ---- 污染与地质灾害 ----
    if "污染" in name:
        if check is None or check.pollution_pass is None:
            return MANUAL, "尚未完成限制性条件判定"
        return ((OK, "限制性判定：土壤污染一项通过") if
check.pollution_pass else
                (BAD, "限制性判定：土壤污染一项未通过"))
    if "地质灾害" in name:
        if check is None or check.geo_pass is None:
            return MANUAL, "尚未完成限制性条件判定"
        if not check.geo_pass:
            return BAD, "限制性判定：存在地质灾害"
        if "发育程度" in name:
            return MANUAL, "系统只记录「有无地质灾害」，未记录发育程度分级"
        return OK, "限制性判定：地质灾害一项通过"
    # ---- 责任主体 ----
    if "责任主体" in name:
```

```
        missing = ctx["patch"].responsible_entity_missing
        if missing is None:
            return MANUAL, "图斑档案未填写责任主体是否缺失"
        if missing:
            return OK, "图斑档案记录责任主体缺失，按文件属政府主导治理"
        if "政府" in name:
            return MANUAL, "责任主体未缺失，是否符合「政府」要求需人工核实"
        return OK, "图斑档案记录责任主体明确"
    # ---- 适宜性分级 / 指数 ----
    if "适宜性分级" in name or "适宜性评价" in name:
        # 文件只要求「做过自然恢复适宜性分级」，系统已经给出等级
        if ctx["grade"] is None:
            return MANUAL, "尚未执行定量评价"
        return OK, "系统已给出适宜性等级：%s（F=%.4f）" % (ctx["grade"],
    ctx["score"])
    if "适宜性指数" in name:
        # 指数是要跟阈值比的：文件没给具体阈值时不能替它下结论
        if ctx["grade"] is None:
            return MANUAL, "尚未执行定量评价"
        t = _num(threshold)
        if t is None:
            return MANUAL, ("文件未给出具体阈值，系统评价因子
    F=%.4f（%s），需人工比对"
                            % (ctx["score"], ctx["grade"]))
        r = _cmp(ctx["score"], op, t)
        if r is None:
            return MANUAL, "系统评价因子 F=%.4f" % ctx["score"]
        return (OK if r else BAD), ("系统评价因子 F=%.4f，文件要求 %s
    %s"
                                    % (ctx["score"], op,
    threshold))
    # ---- 其它定性要求 ----
    return MANUAL, "系统内无对应数据，需人工核实"
def compare(patch_id):
    """生成某图斑的评价结论与当地标准 / 政策文件的对照结果。"""
    patch = db.session.get(MinePatch, patch_id)
    if patch is None:
        return None
    province = province_of(patch.admin_region)
    out = {
        "patch": patch,
        "region": patch.admin_region or "",
        "province": province,
        "covered": False,
        "message": "",
        "standards": [],
        "totals": {OK: 0, BAD: 0, MANUAL: 0},
        "necessary": {"符合": 0, "共计": 0},
    }
    if province is None:
```

```
        out["message"] = ("未收录该省份的地方标准与政策文件"
                          "（图斑所在行政区：%s）" %
    (patch.admin_region or "未填写"))
        return out
    standards = (Standard.query.filter_by(province=province)
                 .order_by(Standard.category.desc(),
    Standard.name).all())
    if not standards:
        out["message"] = "未收录该省份的地方标准与政策文件（%s）" % province
        return out
    ctx = _context(patch)
    out["covered"] = True
    for s in standards:
        rules = (StandardRule.query.filter_by(standard_id=s.
    id)
                 .order_by(StandardRule.id).all())
        rows = []
        for r in rules:
            verdict, evidence = _judge(r.item, r.operator,
    r.threshold, ctx)
            rows.append({
                "item": r.item, "operator": r.operator,
    "threshold": r.threshold,
                "condition_type": r.condition_type or "",
    "region": r.region,
                "note": r.note, "verdict": verdict, "tag":
    TAG_CLASS[verdict],
                "evidence": evidence,
            })
            out["totals"][verdict] = out["totals"].get(verd
    ct, 0) + 1
            if (r.condition_type or "") == "必要条件":
                out["necessary"]["共计"] += 1
                if verdict == OK:
                    out["necessary"]["符合"] += 1
        out["standards"].append({
            "name": s.name, "doc_number": s.doc_number,
    "category": s.category,
            "summary": s.summary, "rules": rows,
            "counts": {k: sum(1 for x in rows if
    x["verdict"] == k)
                       for k in (OK, BAD, MANUAL)},
        })
    return out
# ==== app/services/unit_config.py ====
# -*- coding: utf-8 -*-
"""标准化参数与必填项：让界面上能改的东西真正参与计算。
以前有个说不通的地方：图斑的「参照生态系统基准」、指标定义里的「理想取值」
和「启用单元」在界面上都能改、也能落库，**但计算链只读代码常量**
（`app/utils/constants.py`），从不查这两张表。于是报告里写着「基准 60%」，
```

实际按常量 75% 算，同一份报告两个口径。
现在统一从这里取：
- 基准值（植被覆盖度、Shannon）来自图斑的 `reference_ecosystems`；
- 各指标的理想取值来自 `indicators.ideal_value`（能解析成数字的才覆盖，
  像「5~9」「25° 分界」这种区间 / 规则型仍按代码里的算法处理）；
- 启用单元来自 `indicators.enabled_units`（岩质边坡无土壤、平台无地形）。
库里没有行、或值不是数字时一律回落到代码常量 —— 所以历史数据与
`验证数据/` 里的 8 个算例结果不会因为这次改动而变化。
"""
from app.models import Indicator, ReferenceEcosystem
from app.utils.constants import (
    UNIT_TYPES, IDEAL_SOIL_RATIO, IDEAL_THICKNESS,
    IDEAL_ORGANIC,
    IDEAL_COVERAGE, IDEAL_SHANNON, HEAVY_METAL_LIMIT,
    HEAVY_METALS,
)
# 二级指标编码 → 标准化参数名（同时也是 IndicatorData 的字段名）
NUMERIC_IDEALS = {
    "soil_ratio": ("soil_ratio", IDEAL_SOIL_RATIO),
    "thickness": ("thickness", IDEAL_THICKNESS),
    "organic_matter": ("organic_matter", IDEAL_ORGANIC),
    "coverage": ("coverage", IDEAL_COVERAGE),
    "shannon": ("shannon", IDEAL_SHANNON),
}
# 重金属编码 → 金属名
METAL_CODES = {"hm_" + m: m for m in HEAVY_METALS}
# 必填的实测字段（污染指标按方法约定允许「未检测」，故不在必填之列）
REQUIRED_CODES = ["soil_ratio", "thickness", "ph",
    "organic_matter",
                  "coverage", "shannon", "slope", "aspect"]
# 字段 → 界面上的中文名，用于提示「缺什么」
FIELD_LABELS = {
    "soil_ratio": "土石比", "thickness": "土层厚度", "ph": "pH",
    "organic_matter": "有机质", "coverage": "植被覆盖度",
    "shannon": "生物多样性（Shannon）", "slope": "坡度", "aspect":
    "坡向",
}
def _numeric(text):
    """把理想取值的文本解析成数字；区间、规则型返回 None。"""
    if text is None:
        return None
    try:
        return float(str(text).strip())
    except (TypeError, ValueError):
        return None
class UnitSettings:
    """一个图斑上下文的标准化参数。空对象表示「全部按代码常量」。"""
    def __init__(self, params=None, enabled=None):
        self.params = params or {}          # 覆盖用的标准值
        self.enabled = enabled or {}        # 单元类型 →

```
    启用的二级指标编码集合
    def ideal(self, code, fallback):
        """取某指标的理想取值：库里有数字就用库里的，否则用常量。"""
        value = self.params.get(code)
        return fallback if value is None else value
    def group_enabled(self, unit_type, l1_name, l1_code):
        """该单元类型下某个一级指标是否启用（库里没配就按代码常量）。"""
        codes = self.enabled.get(unit_type)
        if codes is None:
            return None                                  # 交给调用方按常量判断
        return l1_code in codes
    def enabled_codes(self, unit_type):
        """该单元类型启用的全部二级指标编码；库里没配返回 None。"""
        return self.enabled.get(unit_type)
    def required_codes(self, unit_type):
        """该单元类型必填的实测字段。
        按「启用的二级指标」推导：岩质边坡不启用土壤，平台不启用地形。
        库里没配该项时用代码常量里的结构性规则。
        """
        from app.utils.constants import
    UNIT_ENABLED_INDICATORS
        codes = self.enabled.get(unit_type)
        if codes is None:
            groups = UNIT_ENABLED_INDICATORS.get(unit_type,
    ())
            l2_by_group = {
                "土壤指标": ["soil_ratio", "thickness", "ph",
    "organic_matter"],
                "植被指标": ["coverage", "shannon"],
                "地形指标": ["slope", "aspect"],
            }
            codes = {c for g in groups for c in
    l2_by_group.get(g, [])}
        return [c for c in REQUIRED_CODES if c in codes]
    def missing_fields(self, unit_type, data):
        """返回该单元缺哪些必填项（中文名列表）。"""
        missing = []
        for code in self.required_codes(unit_type):
            value = data.get(code)
            if value is None or (isinstance(value, str) and
    value.strip() == ""):
                missing.append(FIELD_LABELS.get(code, code))
        return missing
    def undetected_metals(self, data):
        """按方法约定，未检测的重金属按「未受污染」计入；这里列出是哪些。"""
        measured = data.get("heavy_metals") or {}
        return [m for m in HEAVY_METALS if measured.get(m)
    is None]
def for_patch(patch_id):
    """读某个图斑的标准化参数；库里有什么就用什么，没有就用常量。
```

```
    优先级：图斑自己的参照生态系统基准  **高于**  指标定义里的全局理想取值  ——
    前者说的是「这个图斑的参照生态系统是什么样」，后者是「一般情况下的理想值」，
    越具体的越应该说了算。两者都优先于代码常量。
    """
    params = {}
    enabled = {}
    for ind in Indicator.query.all():
        code = ind.code
        if code in NUMERIC_IDEALS:
            value = _numeric(ind.ideal_value)
            if value is not None:
                params[code] = value
        elif code in METAL_CODES:
            #  重金属的「理想值」存的是限值，作为越小越优的极值使用
            value = _numeric(ind.ideal_value)
            if value is not None:
                params[code] = value
        units = (ind.enabled_units or "").strip()
        if units:
            for unit in UNIT_TYPES:
                if unit in units:
                    enabled.setdefault(unit,
    set()).add(code)
    #  再用该图斑的参照生态系统基准覆盖：它是「这个图斑的参照生态系统」，
    #  比指标定义里的全局理想取值更具体，应该它说了算。
    ref = ReferenceEcosystem.query.filter_by(patch_id=patch_
    id).first()
    if ref is not None:
        if ref.coverage_benchmark is not None:
            params["coverage"] = float(ref.coverage_benchmar
    k)
        if ref.shannon_benchmark is not None:
            params["shannon"] = float(ref.shannon_benchmark)
    return UnitSettings(params=params, enabled=enabled)
def metal_limits(settings):
    """重金属的极值表：库里有配置的用库里的，其余用常量。"""
    limits = dict(HEAVY_METAL_LIMIT)
    for code, metal in METAL_CODES.items():
        value = settings.params.get(code) if settings else
    None
        if value is not None:
            limits[metal] = value
    return limits
#  ==== app/templates/standard/_check_block.html ====
{#  结论对照内容块：标准知识库对照页与评价结果页共用同一份渲染  #}
{%  if  check  %}
<h3>与地方标准  /  政策文件的对照</h3>
{%  if  not  check.covered  %}
<p  class="muted">{{  check.message  }}。标准知识库只作文件储备与比对，不参与评价计算；
  如需对照，请先在「标准知识库」中收录该省份的标准或政策文件。</p>
```

```
{% else %}
<p class="muted">
  图斑所在行政区：{{ check.region or "未填写" }}（对照 {{ check.province }} 的标准与政策文件）。
  对照只是把文件里的判定项逐条映射到系统已有数据：共 {{ check.totals["符合"] + check.totals["不符合"] + check.totals["待人工判断"] }} 条，
  符合 <strong>{{ check.totals["符合"] }}</strong> 条、不符合 <strong>{{ check.totals["不符合"] }}</strong> 条、
  待人工判断 <strong>{{ check.totals["待人工判断"] }}</strong> 条。
  文件里的定性要求（如「地形地貌与周边协调」）系统没有对应数据，一律标为待人工判断，不代为结论。
</p>
{% for s in check.standards %}
<h4 style="margin:14px 0 6px;">{{ s.name }}{% if s.doc_number %} <span class="muted mono">{{ s.doc_number }}</span>{% endif %}</h4>
<table>
  <caption class="muted">{{ s.category }}｜符合 {{ s.counts["符合"] }} 条、不符合 {{ s.counts["不符合"] }} 条、待人工判断 {{ s.counts["待人工判断"] }} 条</caption>
  <thead>
    <tr>
      <th scope="col">文件判定项</th>
      <th scope="col">条件</th>
      <th scope="col">类型</th>
      <th scope="col">对照结果</th>
      <th scope="col">依据（系统内的对应数据）</th>
    </tr>
  </thead>
  <tbody>
    {% for r in s.rules %}
    <tr>
      <td style="text-align:left;">{{ r.item }}</td>
      <td class="mono nowrap">{{ r.operator }} {{ r.threshold or "" }}</td>
      <td>{{ r.condition_type or "—" }}</td>
      <td><span class="tag {{ r.tag }}">{{ r.verdict }}</span></td>
      <td style="text-align:left;">{{ r.evidence }}</td>
    </tr>
    {% endfor %}
  </tbody>
</table>
{% endfor %}
<p class="muted">
  对照结论仅供评价与报送时参考：符合全部必要条件的，说明评价结论与文件要求一致；
  出现「不符合」或「待人工判断」的，需由评价人员结合现场情况核定后再报送。
</p>
{% endif %}
{% endif %}
#  ==== app/templates/standard/check.html ====
```

```
{% extends "base.html" %}
{% block title %}结论对照 · {{ check.patch.name }}{% endblock %}
{% block content %}
<div class="card">
  <h2>结论对照 · {{ check.patch.name }}</h2>
  {% include "standard/_check_block.html" %}
  <p class="actions" style="margin-top:16px;">
    <a class="btn btn--ghost" href="/standard/check">返回图斑列表</a>
    <a class="btn btn--ghost" href="/standard/">返回标准列表</a>
  </p>
</div>
{% endblock %}
# ==== app/templates/standard/check_index.html ====
{% extends "base.html" %}
{% block title %}结论对照{% endblock %}
{% block content %}
<div class="card">
  <h2>评价结论与地方标准 / 政策文件的对照</h2>
  <p class="muted">
    标准知识库不参与评价计算，只在评价完成后回答一个问题：
    本次评价结论是否符合图斑所在省份的文件要求。选择一个图斑查看逐条对照。
  </p>
  {% if not rows %}
  <p class="muted">暂无矿山图斑。请先在「项目与矿山档案」中建立图斑并录入数据。</p>
  {% else %}
  <table>
    <thead>
      <tr>
        <th scope="col">矿山图斑</th>
        <th scope="col">所在行政区</th>
        <th scope="col">已收录该省份文件</th>
        <th scope="col">操作</th>
      </tr>
    </thead>
    <tbody>
      {% for row in rows %}
      <tr>
        <td style="text-align:left;">{{ row.patch.name }}</td>
        <td style="text-align:left;">{{ row.patch.admin_region or "—" }}</td>
        <td>
          {% if row.province %}<span class="tag tag--ok">{{ row.province }}</span>
          {% else %}<span class="tag tag--muted">未收录</span>{% endif %}
        </td>
        <td><a class="link-btn" href="/standard/check/{{ row.patch.id }}">查看对照</a></td>
```

```
        </tr>
        {% endfor %}
      </tbody>
    </table>
    {% endif %}
    <p class="actions" style="margin-top:16px;">
      <a class="btn btn--ghost" href="/standard/">返回标准列表</a>
    </p>
  </div>
{% endblock %}
# ==== app/utils/__init__.py ====
# -*- coding: utf-8 -*-
# ==== app/utils/parsing.py ====
# -*- coding: utf-8 -*-
"""表单数值的安全解析。
背景：页面上的数字输入框原来写成 `float(request.form.get("area"))`，
用户填「约 3.5」或全角数字就会撞上 ValueError —— 页面上是默认英文 500 页。
真人演示时这是最容易踩到的一类。统一走这里：
没填返回 None，填了但不是数字抛 FieldError，让视图 flash 一句中文再回退。
"""
import math
class FieldError(ValueError):
    """表单里某个字段填得不对，携带给用户看的中文提示。"""
def parse_float(raw, field, *, minimum=None, maximum=None):
    """把表单值解析成 float。
    raw       表单原始值（可能是 None / "" / "abc" / "1e400"）
    field     字段名，用于拼提示
    minimum / maximum   可选的取值范围，越界同样抛 FieldError
    """
    if raw is None:
        return None
    text = str(raw).strip()
    if text == "":
        return None
    try:
        value = float(text)
    except (TypeError, ValueError):
        raise FieldError("「%s」请填数字" % field)
    # nan / inf 能通过 float()，但存进库里会把标准化算成满分，必须挡住
    if not math.isfinite(value):
        raise FieldError("「%s」请填正常的数字" % field)
    if minimum is not None and value < minimum:
        raise FieldError("「%s」不能小于 %g" % (field, minimum))
    if maximum is not None and value > maximum:
        raise FieldError("「%s」不能大于 %g" % (field, maximum))
    return value
def parse_int(raw, field, *, default=None, minimum=None,
              maximum=None):
    """解析整数，用于查询参数与计数类字段。"""
    if raw is None:
```

```
            return default                                                  13757
        text = str(raw).strip()                                             13758
        if text == "":                                                      13759
            return default                                                  13760
        try:                                                                13761
            value = int(text)                                               13762
        except (TypeError, ValueError):                                     13763
            raise FieldError("「%s」请填整数" % field)                        13764
        if minimum is not None and value < minimum:                         13765
            value = minimum                                                 13766
        if maximum is not None and value > maximum:                         13767
            value = maximum                                                 13768
        return value                                                        13769
# ==== app/utils/timefmt.py ====                                            13770
# -*- coding: utf-8 -*-                                                     13771
"""时间显示：库里一律存 UTC，展示时换成服务器本地时间。                          13772
为什么不在写库时直接用本地时间：一个系统里时间基准只能有一个。                  13773
`app/models.py` 里所有 `created_at` / `checked_at` 都用                     13774
    `datetime.utcnow`，                                                     13775
存的是 UTC；如果写库改成本地时间，已经存进库的几千条记录会新旧混着差 8 小时。  13776
所以统一在「展示」这一层换算，老数据不用动。                                  13777
用法（Jinja）：                                                             13778
    {{ row.created_at | localdt }}                          →               13779
    2026-09-26 23:33                                                        13780
    {{ row.created_at | localdt("%Y-%m-%d %H:%M:%S") }} →                   13781
    2026-09-26 23:33:28                                                     13782
"""                                                                         13783
from datetime import datetime, timezone                                     13784
DEFAULT_FORMAT = "%Y-%m-%d %H:%M"                                           13785
def to_local(value, fmt=DEFAULT_FORMAT, empty="—"):                         13786
    """把 UTC 时间换算成服务器本地时间并格式化。                              13787
    传进来的值按「无时区即 UTC」处理；带时区的原样换算，便于以后改用          13788
    带时区的时间戳。空值返回占位符，免得页面上出现 None。                    13789
    """                                                                     13790
    if value is None:                                                       13791
        return empty                                                        13792
    if value.tzinfo is None:                                                13793
        value = value.replace(tzinfo=timezone.utc)                          13794
    return value.astimezone().strftime(fmt)                                 13795
def now_utc():                                                              13796
    """取当前 UTC 时间（带时区），用于只作展示、不落库的时间戳。"""          13797
    return datetime.now(timezone.utc)                                       13798
def utcnow():                                                               13799
    """取当前 UTC 时间（无时区），用于数据库默认值。                          13800
    `datetime.utcnow()` 在 Python 3.12 已标记弃用，导入时会刷告警；          13801
    这里用带时区的 now() 再摘掉时区，得到的仍是 naive-UTC，行为一致。        13802
    """                                                                     13803
    return datetime.now(timezone.utc).replace(tzinfo=None)                  13804
# ==== app/views/__init__.py ====                                           13805
# -*- coding: utf-8 -*-                                                     13806
```